Summarize

Chapter 12. Transactional Sagas

Thursday, March 31, 16:55



Austen showed up at Logan's office late on a windy Thursday afternoon. "Addison just sent me over here to ask you about some horror story?"

Logan stopped and looked up. "Is that a description of whatever crazy extreme sport you're doing this weekend? What is it this time?"

"It's late spring, so a bunch of us are going ice skating on the thawing lake. We're wearing body suits, so it's really a combination of skating and swimming. But that's not what Addison meant at all. When I showed Addison my design for the Ticketing workflow, I was immediately instructed to come to you and tell you I've created a horror story."

Logan laughed. "Oh, I see what's going on—you stumbled into the Horror Story saga communication pattern. You designed a workflow with asynchronous communication, atomic transactionality, and choreography, right?"

"How did you know?"

"That's the Horror Story saga pattern, or really, anti-pattern. There are eight generic saga patterns we start from, so it's good to know what they are, because each has a different balance of trade-offs."

The concept of a *saga* in architecture predates microservices, originally concerned with limiting the scope of database locks in early distributed architectures—the paper largely assumed to have coined the concept is from the Proceedings of the 1987 ACM conference. In his book *Microservices Patterns* (Manning Publications) and also outlined in the "Saga Pattern" section of his website, Chris Richardson describes the *saga pattern* for microservices as a sequence of local transactions where each update publishes an event, thus triggering the next update in the sequence. If any of those updates fail, the saga issues a series of compensating updates to undo the prior changes made during the saga.

However, recall from Chapter 2 that this is only one of eight possible types of sagas. In this section, we dive much deeper and look at the inner workings of transactional sagas and how to manage them, particularly when errors occur. After all, since distributed transactions lack atomicity (see "Distributed Transactions"), what makes them interesting is when problems occur.

Transactional Saga Patterns

In Chapter 2, we introduced a matrix that juxtaposed each of the intersecting dimensions when architects must choose how to implement a transactional saga, reproduced in Table 12-1.

Table 12-1. The matrix of dimensional intersections for distributed architectures

Pattern name	Communication	Consistency	Coordination
Epic Saga$^{(sao)}$	Synchronous	Atomic	Orchestrated
Phone Tag Saga$^{(sac)}$	Synchronous	Atomic	Choreographed
Fairy Tale Saga$^{(seo)}$	Synchronous	Eventual	Orchestrated
Time Travel Saga$^{(sec)}$	Synchronous	Eventual	Choreographed
Fantasy Fiction Saga$^{(aao)}$	Asynchronous	Atomic	Orchestrated
Horror Story$^{(aac)}$	Asynchronous	Atomic	Choreographed
Parallel Saga$^{(aeo)}$	Asynchronous	Eventual	Orchestrated
Anthology Saga$^{(aec)}$	Asynchronous	Eventual	Choreographed

We provide whimsical names for each combination, all derived from types of sagas. However, the pattern names exist to help differentiate the possibilities, and we don't want to provide a memorization test to associate a pattern name to a set of characteristics, so we have added a superscript to each saga type indicating the values of the three dimensions listed in alphabetical order (as in Table 12-1). For example, the *Epic Saga$^{(sao)}$ pattern* indicates the values of *synchronous*, *atomic*, and *orchestrated* for communication, consistency, and coordination. The superscripts help you associate names to character sets more easily.

While architects will utilize some of the patterns more than others, they all have legitimate uses and differing sets of trade-offs.

We illustrate each possible communication combination with both a three-dimensional representation of the intersection of the three forces in space along with an example workflow using generic distributed services, which we refer to as *isomorphic diagrams*. These diagrams show interactions between services in the most generic way, toward our goal of showing architect concepts in the simplest form. In each of these diagrams, we use the set of generic symbols shown in Figure 12-1.

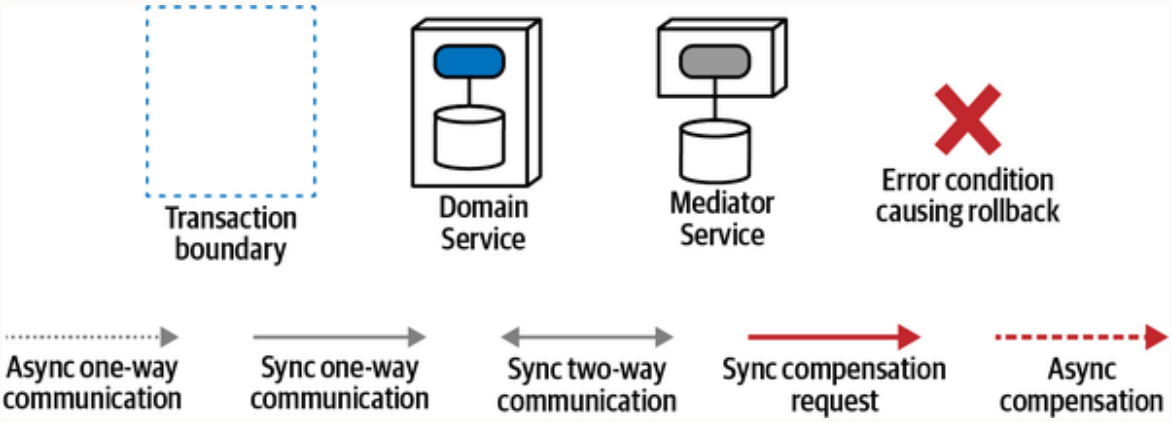


Figure 12-1. Legend for ISO architecture interaction diagrams

For each of the architecture patterns, we do not show every possible interaction, which would become repetitive. Instead, we identify and illustrate the differentiating features of the pattern—what makes its behavior unique among the patterns.

Epic Saga(sao) Pattern

This type of communication is the "traditional" saga pattern as many architects understand it, also called an *Orchestrated Saga* because of its coordination type. Its dimensional relationships appear in Figure 12-2.

This pattern utilizes *synchronous* communication, *atomic* consistency, and *orchestrated* coordination. The architect's goal when choosing this pattern mimics the behavior of monolithic systems—in fact, if a monolithic system were added to this diagram in Figure 12-2, it would be the origin (0, 0, 0), lacking distribution entirely. Thus, this communication style is most familiar with architects and developers of traditional transactional systems.

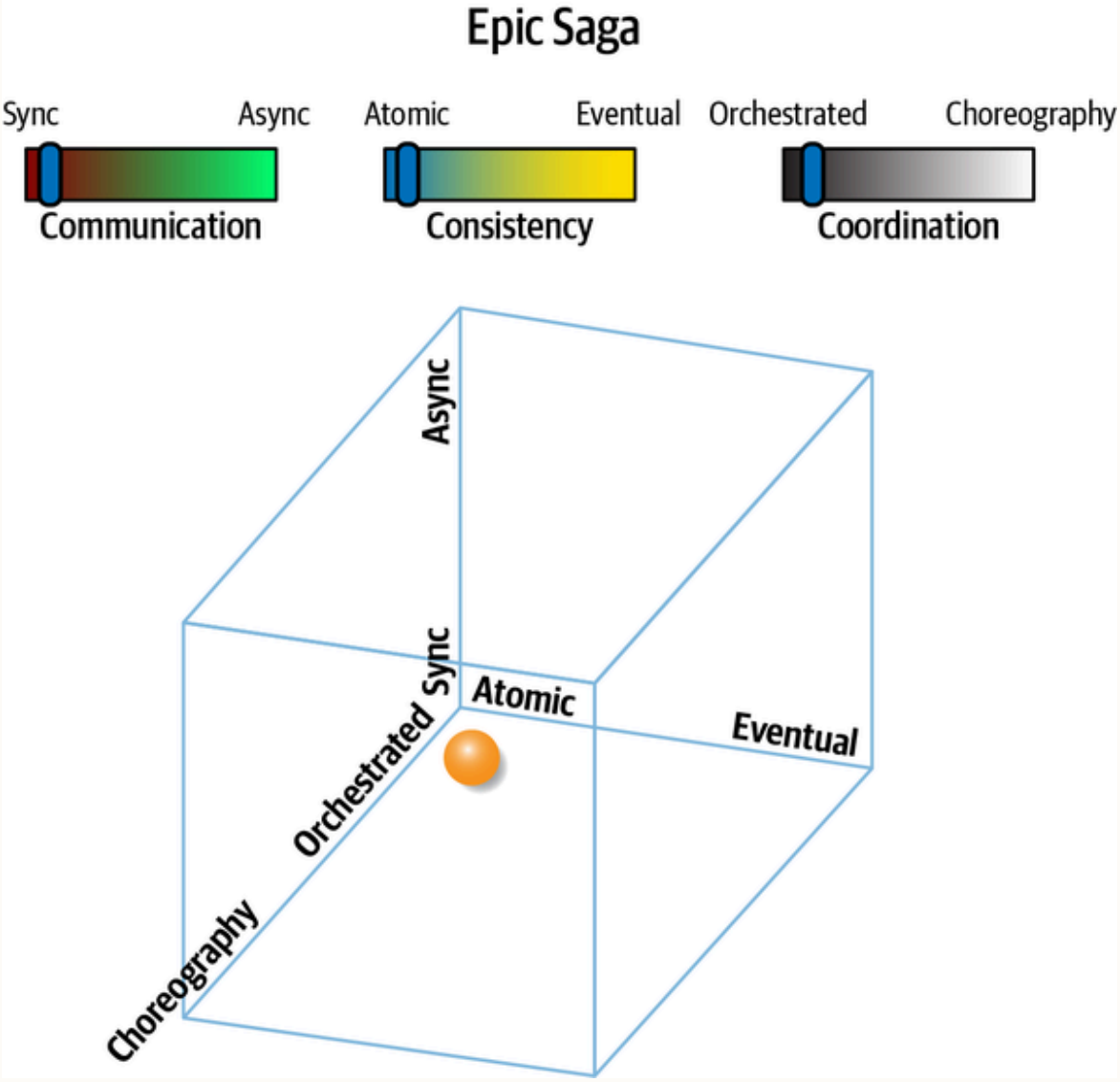


Figure 12-2. The Epic Saga(sao) pattern's dynamic coupling (communication, consistency, coordination) relationships

The isomorphic representation of the Epic Saga(sao) pattern appears in Figure 12-3.

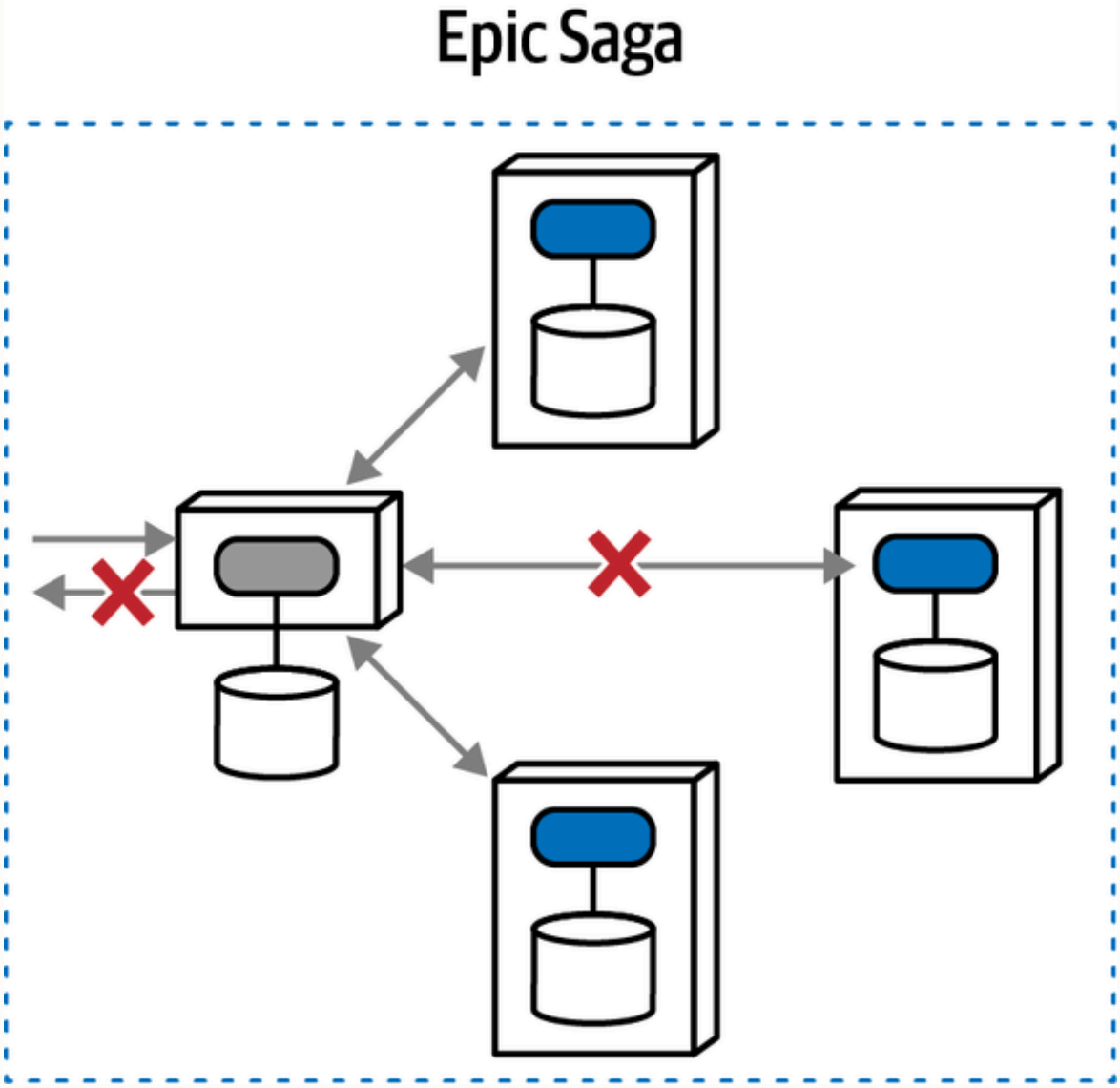


Figure 12-3. The isomorphic communication illustration of the *Epic Saga*(sao) pattern

Here, an *orchestrator* service orchestrates a workflow that includes updates for three services, expected to occur transactionally—either all three calls succeed or none do. If one of the calls fails, they all fail and return to the previous state. An architect can solve this coordination problem in a variety of ways, all complex in distributed architectures. However, such transactions limit the choice of databases and have legendary failure modes.

Many nascent or naive architects trust that, because a pattern exists for a problem, it represents a clean solution. However, the pattern is recognition of only commonality, not solvability. Distributed transactions provide an excellent example of this phenomenon—architects accustomed to modeling transactions in nondistributed systems sometimes believe that moving that capability to the distributed world is an incremental change. However, transactions in distributed architectures present a number of challenges, which become proportionally worse depending on the complexity of the semantic coupling of the problem.

Consider a common implementation of the Epic Saga(sao) pattern, utilizing compensating transactions. A *compensating update* is one that reverses a data write action performed by another service (such as reversing an update, reinserting a previously deleted row, or delet-

ing a previously inserted row) during the course of the distributed transaction scope. While compensating updates attempt to reverse changes in order to bring distributed data sources back to their original state prior to the start of the distributed transaction, they are riddled with complex issues, challenges, and trade-offs.

A compensating transaction pattern assigns a service to monitor the transactional completeness of a request, as shown in Figure 12-4.

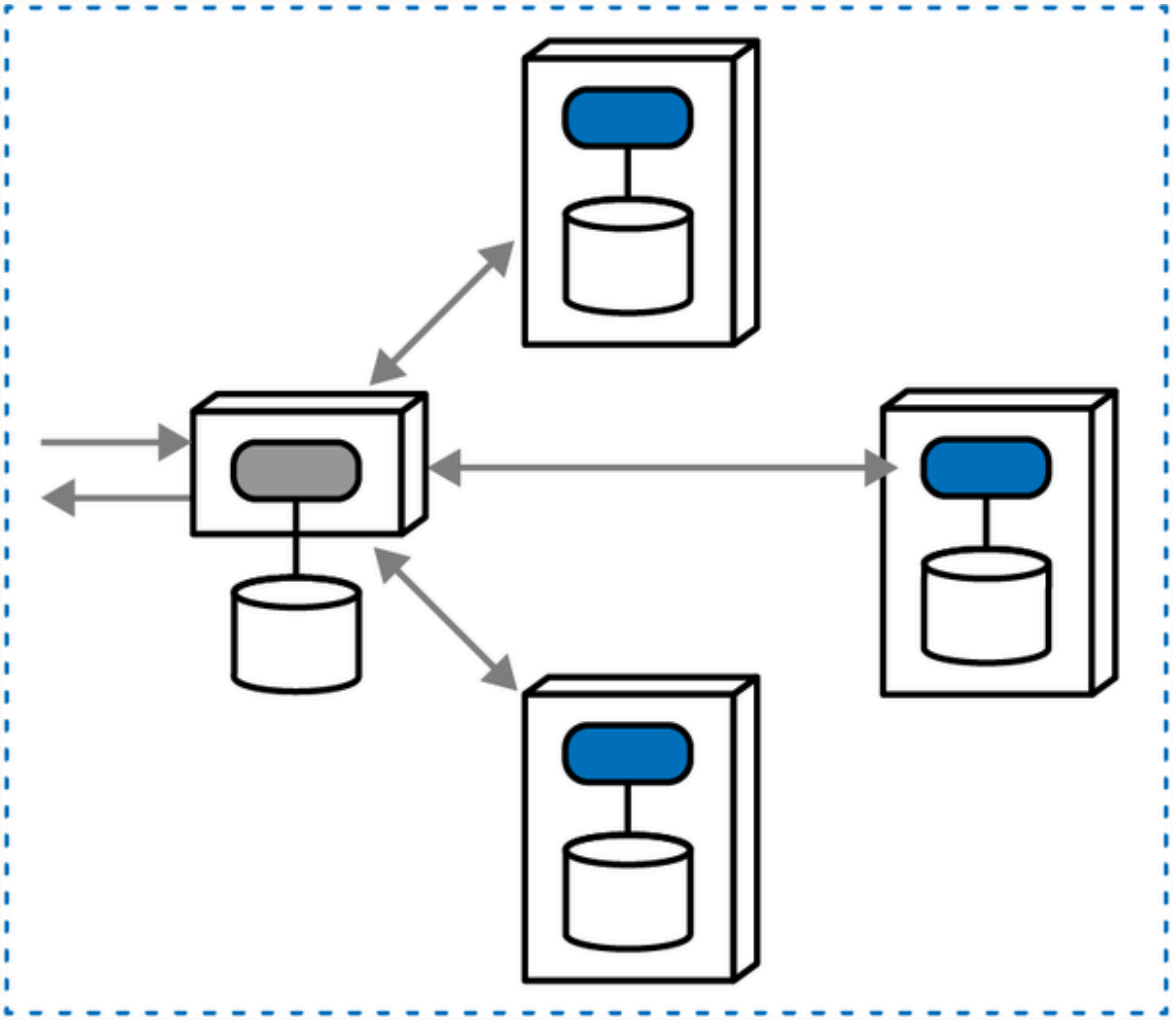


Figure 12-4. A successful orchestrated transactional Epic Saga using a compensating transaction

However, as with many things in architecture, the error conditions cause the difficulties. In a compensating transaction framework, the mediator monitors the success of calls, and issues compensating calls to other services if one or more of the requests fail, as shown in Figure 12-5.

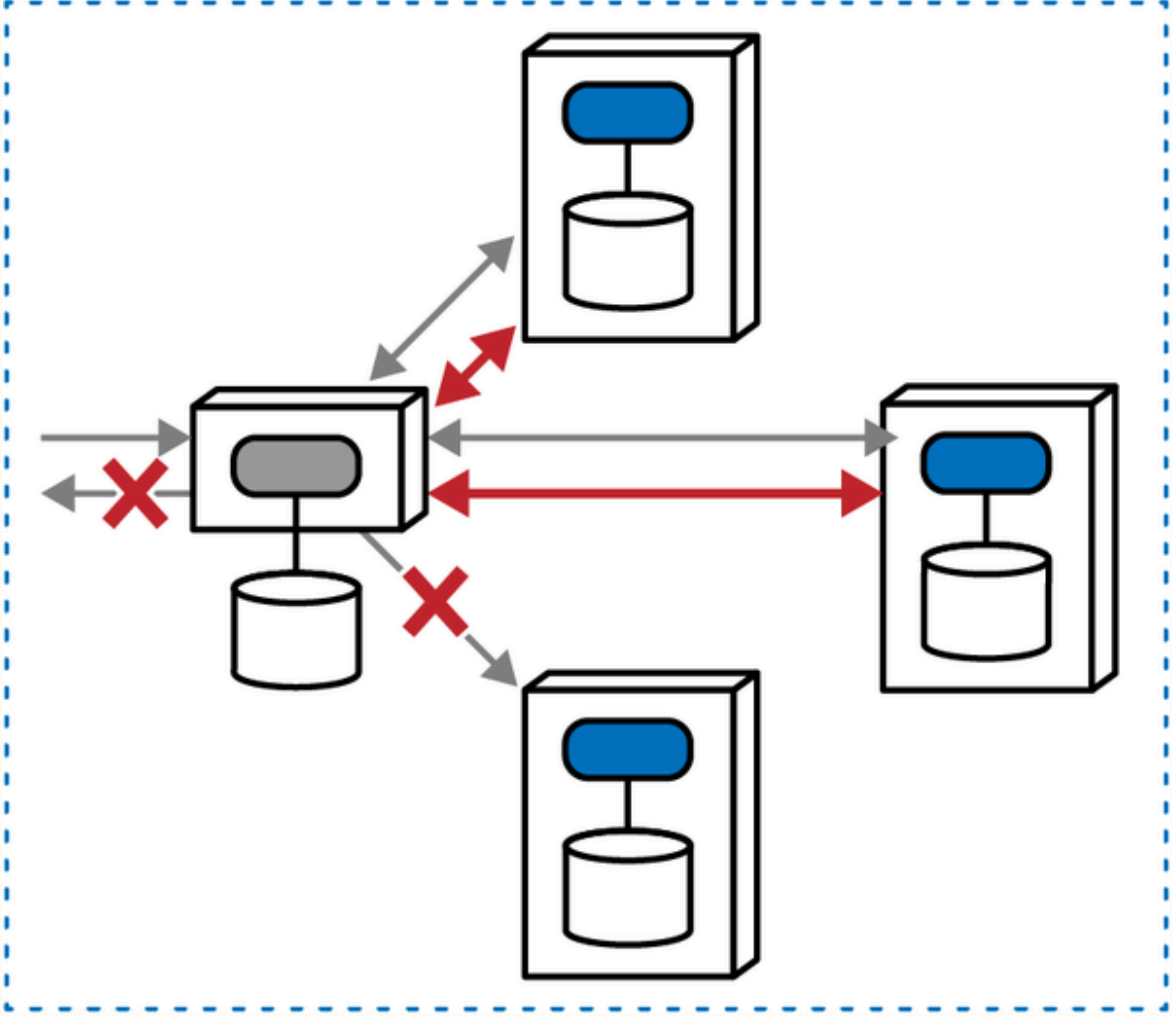


Figure 12-5. When an error occurs, a mediator must send compensating requests to other services

A mediator both accepts requests and mediates the workflow, and synchronous calls to the first two services succeed. However, when trying to make the call to the last service, it fails (from a possibly a wide variety of both domain and operational reasons). Because the goal of the Epic Saga(sao) is atomic consistency, the mediator must utilize compensating transactions and request that the other two services undo the operation from before, returning the overall state to what it was before the transaction started.

This pattern is widely used: it models familiar behavior, and it has a well-established pattern name. Many architects default to the Epic Saga(sao) pattern because it feels familiar to monolithic architectures, combined with a request (sometimes demand) from stakeholders that state changes must synchronize, regardless of technical constraints. However, many of the other dynamic quantum coupling patterns may offer a better set of trade-offs.

The clear advantage of the Epic Saga(sao) is the transactional coordination that mimics monolithic systems, coupled with the clear workflow owner represented via an orchestrator. However, the disadvantages are varied. First, orchestration plus transactionality may have an impact on operational architecture characteristics such as performance, scale, elasticity, and so on—the orchestrator must make sure that all participants in the transaction have succeeded or failed, creating timing bottlenecks. Second, the various patterns used to implement distributed transactionality (such as compensating transactions) succumb to a

wide variety of failure modes and boundary conditions, along with adding inherent complexity via undo operations. Distributed transactions present a host of difficulties and thus are best avoided if possible.

The Epic Saga(sao) pattern features the following characteristics:

Coupling level

This pattern exhibits extremely high levels of coupling across all possible dimensions: synchronous communication, atomic consistency, and orchestrated coordination—it is in fact the most highly coupled pattern in the list. This isn't surprising, as it models the behavior of highly coupled monolithic system communication, but creates a number of issues in distributed architectures.

Complexity level

Error conditions and other intensive coordination added to the requirement of atomicity add complexity to this architecture. The synchronous calls this architecture uses mitigate some of the complexity, as architects don't have to worry about race conditions and deadlocks during calls.

Responsiveness/availability

Orchestration creates a bottleneck, especially when it must also coordinate transactional atomicity, which reduces responsiveness. This pattern uses synchronous calls, further impacting performance and responsiveness. If any of the services are not available or an unrecoverable error occurs, this pattern will fail.

Scale/elasticity

Similar to *responsiveness*, the bottleneck and coordination required to implement this pattern make scale and other operational concerns difficult.

While the Epic Saga(sao) is popular because of familiarity, it creates a number of challenges, both from a design and operational characteristics standpoint, as shown in Table 12-2.

Table 12-2. Ratings for the Epic Saga(sao)

Epic Saga(sao) pattern	Ratings
Communication	Synchronous
Consistency	Atomic
Coordination	Orchestrated
Coupling	Very high
Complexity	Low
Responsiveness/availability	Low
Scale/elasticity	Very low

Fortunately, architects need not default to patterns that, while seemingly familiar, create accidental complexity—a variety of other patterns exist with differing sets of trade-offs. Refer to the "Sysops Squad Saga: Atomic Transactions and Compensating Updates" for a concrete example of the Epic Saga(sao) and some of the complex challenges it presents (and how to address those challenges).

Phone Tag Saga(sac) Pattern

The *Phone Tag Saga(sac) pattern* changes one of the dimensions of the Epic Saga(sao), changing *coordination* from *orchestrated* to *choreographed*; this change is illustrated in Figure 12-6.

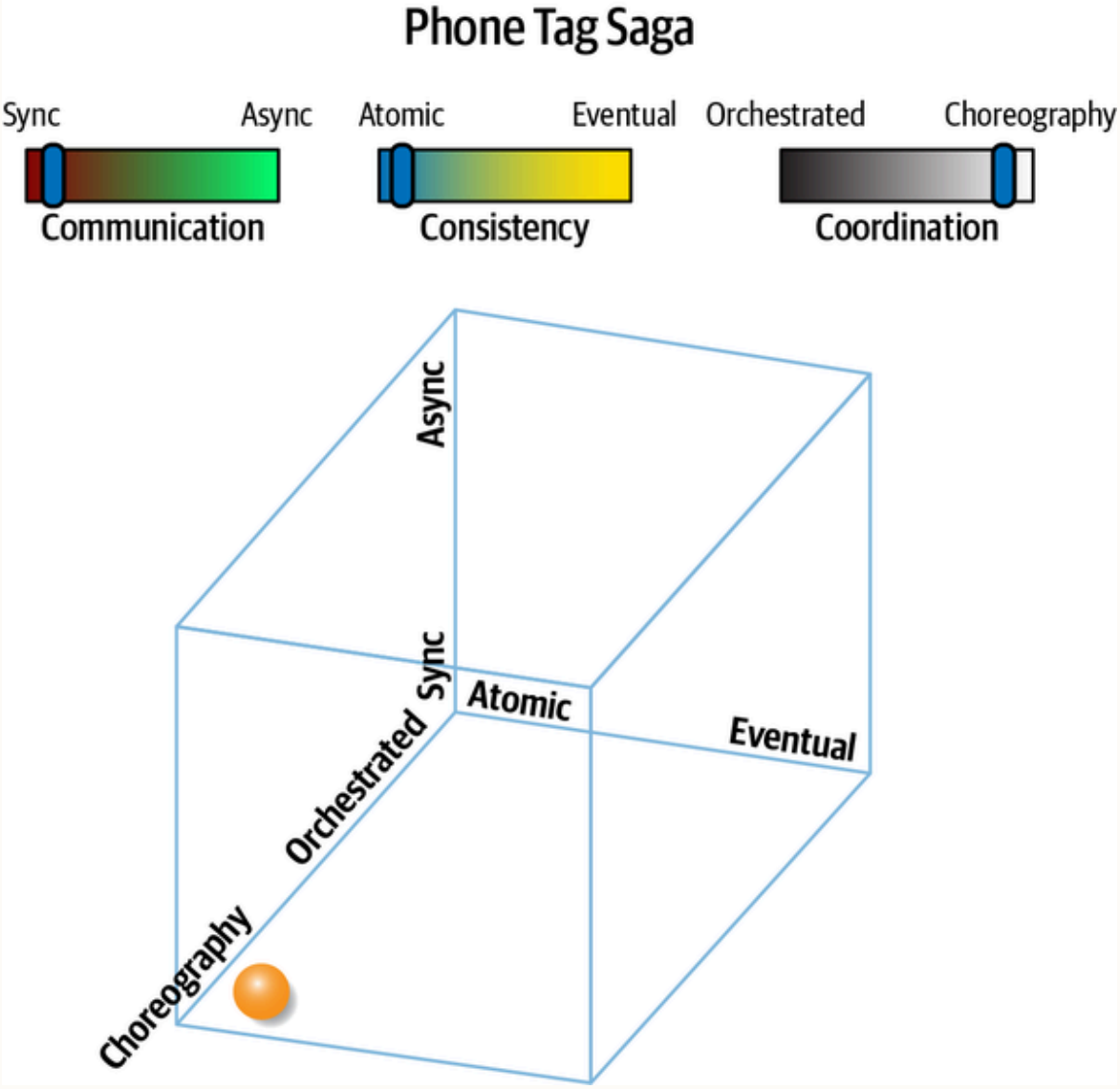


Figure 12-6. The Phone Tag pattern utilizes loosely coupled communication

The pattern name is *Phone Tag* because it resembles a well-known children's game known as *Telephone* in North America: children form a circle, and one person whispers a secret to the next person, who passes it along to the next, until the final version is spoken by the last person. In Figure 12-6, choreography is favored over orchestration, creating the corresponding change in the structural communication shown in Figure 12-7.

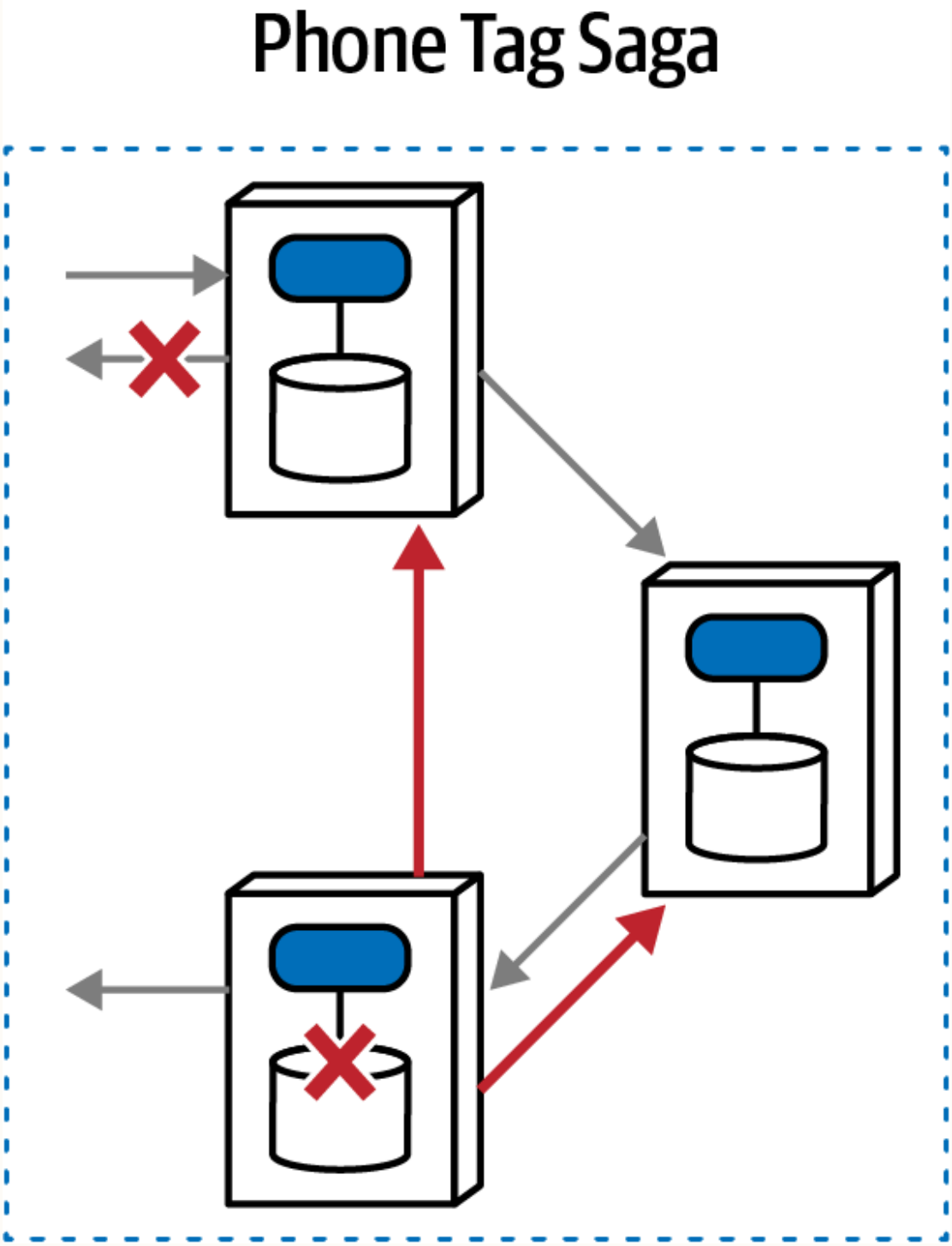


Figure 12-7. Because of a lack of orchestration, each participant must coordinate status

The Phone Tag Saga[(sac)] pattern features atomicity but also choreography, meaning that the architect designates no formal orchestrator. Yet atomicity requires some degree of coordination. In Figure 12-7, the initially called service becomes the coordination point (sometimes called the *front controller*). Once it has finished its work, it passes a request on to the next service in the workflow, which continues until the workflow succeeds. However, if an error condition occurs, each service must have built-in logic to send compensating requests back along the chain.

Because the architectural goal is transactional atomicity, logic to coordinate that atomicity must reside somewhere. Thus, domain services must contain more logic about the workflow context they participate within, including error handling and routing. For complex workflows, the *front controller* in this pattern will become as complex as most mediators, reduc-

ing the appeal and applicability of this pattern. Thus, this pattern is commonly used for simple workflows that need higher scale, but with a potential performance impact.

How does choreography versus orchestration improve operational architecture characteristics like scale? Using choreography even with synchronous communication cuts down on bottlenecks—in nonerror conditions, the last service in the workflow can return the result, allowing for higher throughput and fewer choke points. Performance for happy path workflows can be faster than in an Epic Saga(sao) because of lack of coordination. However, error conditions will be much slower without a mediator—each service must unwind the call chain, which also increases coupling between services.

Generally, the Phone Tag Saga(sac) offers slightly better scale than the Epic Saga(sao) because of the lack of a mediator, which can sometimes become a limiting bottleneck. However, this pattern also features lower performance for error conditions and other workflow complexities—without a mediator, the workflow must be resolved via communication between services, which impacts performance.

A nice feature of nonorchestrated architectures is the lack of a coupling singularity, a single place the workflow couples to. Even though this pattern utilizes synchronous requests, fewer wait conditions for happy path workflows exist, allowing for higher scale. In general, reducing coupling increases scale.

With the improved scalability brought about because of a lack of orchestration comes the increased complexity of the domain services to manage the workflow concerns in addition to their nominal responsibility. For complex workflows, increased complexity and interservice communication may drive architects back toward orchestration and its trade-offs.

The Phone Tag Saga(sac) has a fairly rare combination of features—generally, if an architect chooses *choreography*, they also choose *asynchronicity*. However, in some cases where an architect might choose this combination instead: synchronous calls ensure that each domain service completes its part of the workflow before invoking the next, eliminating race conditions. If error conditions are easy to resolve, or domain services can utilize idempotence and retries, then architects can build higher parallel scale using this pattern compared to an Epic Saga(sao).

The Phone Tag Saga(sac) pattern has the following characteristics:

Coupling level

This pattern relaxes one of the coupling dimensions of the Epic Saga(sao) pattern, utilizing a choreographed rather than orchestrated workflow. Thus, this pattern is

slightly less coupled, but with the same transactional requirement, meaning that the complexity of the workflow must be distributed between the domain services.

Complexity level

This pattern is significantly more complex than the Epic Saga(sao); complexity in this pattern rises linearly proportionally to the semantic complexity of the workflow: the more complex the workflow, the more logic must appear in each service to compensate for lack of orchestrator. Alternatively, an architect might add workflow information to the messages themselves as a form of *stamp coupling* (see "Stamp Coupling for Workflow Management") to maintain state but adding to the overhead context required by each service.

Responsiveness/availability

Less orchestration generally leads to better responsiveness, but error conditions in this pattern become more difficult to model without an orchestrator, requiring more coordination via callbacks and other time-consuming activities.

Scale/elasticity

Lack of orchestration translates to fewer bottlenecks, generally increasing scalability, but only slightly. This pattern still utilizes tight coupling around two of the three dimensions, so scalability isn't a highlight, especially if error conditions are common.

The ratings for the Phone Tag Saga(sac) appear in Table 12-3.

Table 12-3. Ratings for the Phone Tag Saga(sac)

Phone Tag Saga(sac)	Ratings
Communication	Synchronous
Consistency	Atomic
Coordination	Choreographed
Coupling	High
Complexity	High
Responsiveness/availability	Low
Scale/elasticity	Low

The Phone Tag Saga(sac) pattern is better for simple workflows that don't have many common error conditions. While it offers a few better characteristics than the Epic Saga(sao), the complexity introduced by lack of an orchestrator offsets many of the advantages.

Fairy Tale Saga(seo) Pattern

Typical fairy tales provide happy stories with easy-to-follow plots, thus the name *Fairy Tale Saga(seo)*, which utilizes synchronous communication, eventual consistency, and orchestration, as shown in Figure 12-8.

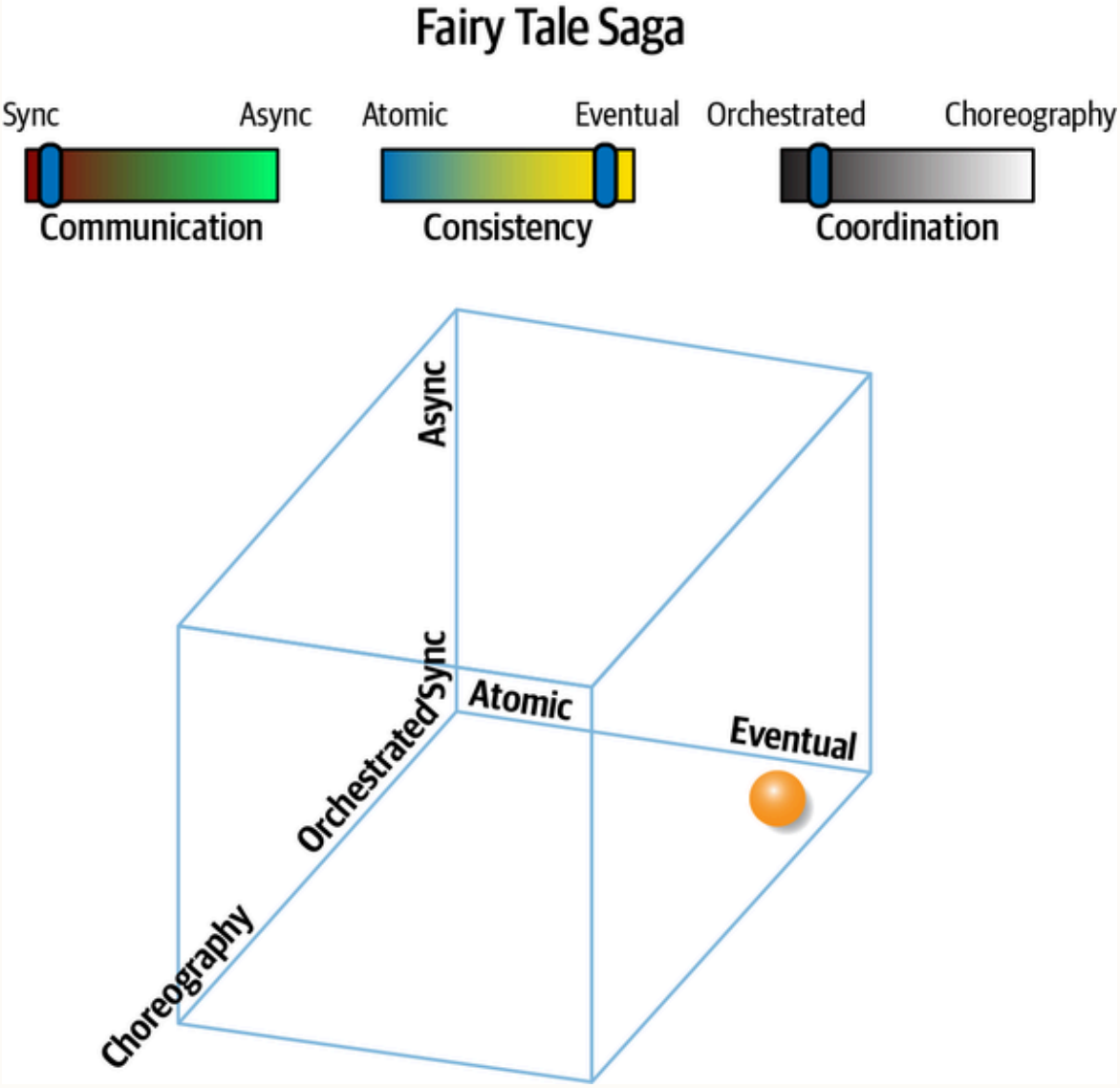


Figure 12-8. The Fairy Tale Saga(seo) illustrates eventual consistency

This communication pattern relaxes the difficult atomic requirement, providing many more options for architects to design systems. For example, if a service is down temporarily, eventual consistency allows for caching a change until the service restores. The communication structure for the Fairy Tale Saga(seo) is illustrated in Figure 12-9.

In this pattern, an orchestrator exists to coordinate request, response, and error handling. However, the orchestrator isn't responsible for managing transactions, which each domain service retains responsibility for (for examples of common workflows, see Chapter 11). Thus the orchestrator can manage compensating calls, but without the requirement of occurring within an active transaction.

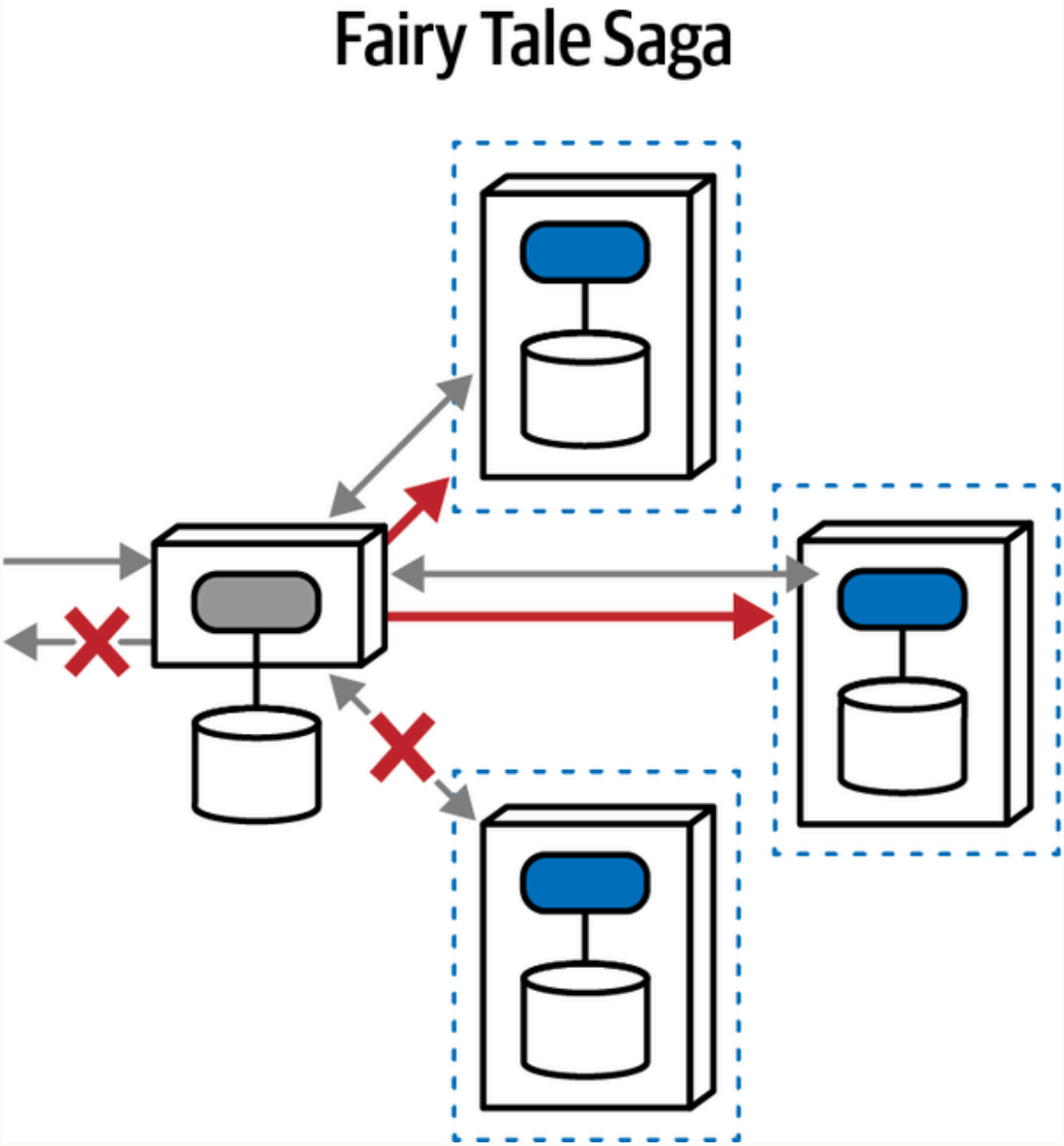


Figure 12-9. Isomorphic illustration of a Fairy Tale interaction

This is a much more attractive pattern and appears commonly in many microservices architectures. Having a mediator makes managing workflows easier, synchronous communication is the easier of the two choices, and eventual consistency removes the most difficult coordination challenge, especially for error handling.

The biggest appealing advantage of the Fairy Tale Saga(seo) is the lack of holistic transactions. Each domain service manages its own transactional behavior, relying on eventual consistency for the overall workflow.

Compared to many other patterns, this pattern generally exhibits a good balance of trade-offs:

Coupling level

The Fairy Tale Saga(seo) features high coupling, with two of the three coupling drivers maximized in this pattern (synchronous communication and orchestrated coordination). However, the worse driver of coupling complexity—transactionality—disap-

pears in this pattern in favor of eventual consistency. The orchestrator must still manage complex workflows, but without the stricture of doing so within a transaction.

Complexity level

Complexity for the Fairy Tale Saga(seo) is quite low; it includes the most convenient options (orchestrated, synchronicity) with the loosest restriction (eventual consistency). Thus the name Fairy Tale Saga(seo)—a simple story with a happy ending.

Responsiveness/availability

Responsiveness is typically better in communication styles of this type because, even though the calls are synchronous, the mediator needs to contain less time-sensitive state about ongoing transactions, allowing for better load balancing. However, true distinctions in performance come with asynchronicity, illustrated in future patterns.

Scale/elasticity

Lack of coupling generally leads to higher scale; removing transactional coupling allows each service to scale more independently.

The ratings for the Fairy Tale Saga(seo) appear in Table 12-4.

Table 12-4. Ratings for the Fairy Tale Saga(seo)

Fairy Tale Saga(seo)	**Ratings**
Communication	Synchronous
Consistency	Eventual
Coordination	Orchestrated
Coupling	High
Complexity	Very low
Responsiveness/availability	Medium
Scale/elasticity	High

If an architect can take advantage of eventual consistency, this pattern is quite attractive, combining the easy moving parts with the fewest scary restrictions, making it a popular

choice among architects.

Time Travel Saga(sec) Pattern

The *Time Travel Saga(sec)* pattern features synchronous communication, and eventual consistency, but choreographed workflow. In other words, this pattern avoids a central mediator, placing the workflow responsibilities entirely on the participating domain services, as illustrated in Figure 12-10.

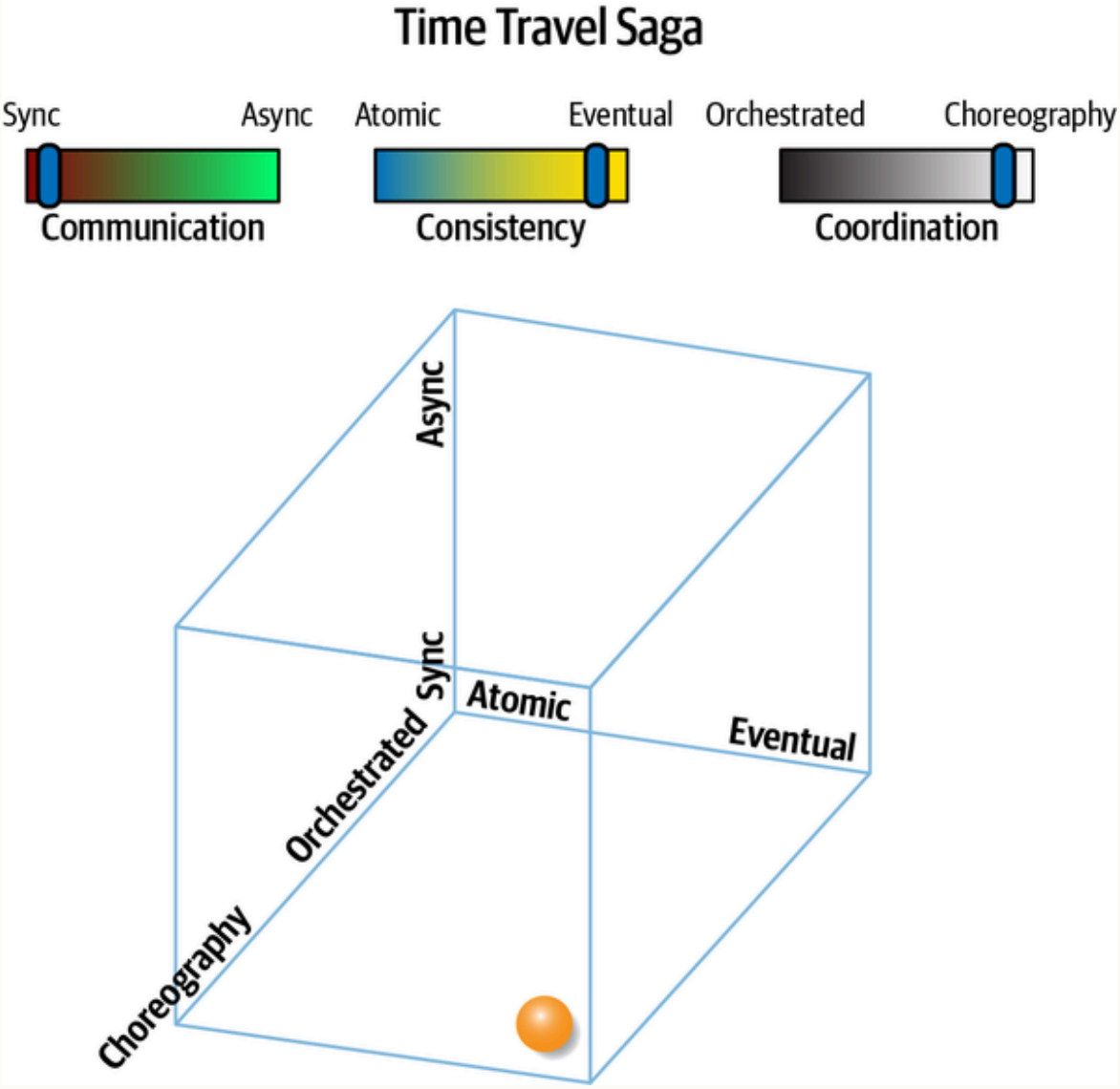


Figure 12-10. The Time Travel Saga(sec) pattern uses two of three decoupling techniques

The structural topology illustrates the lack of orchestration, shown in Figure 12-11.

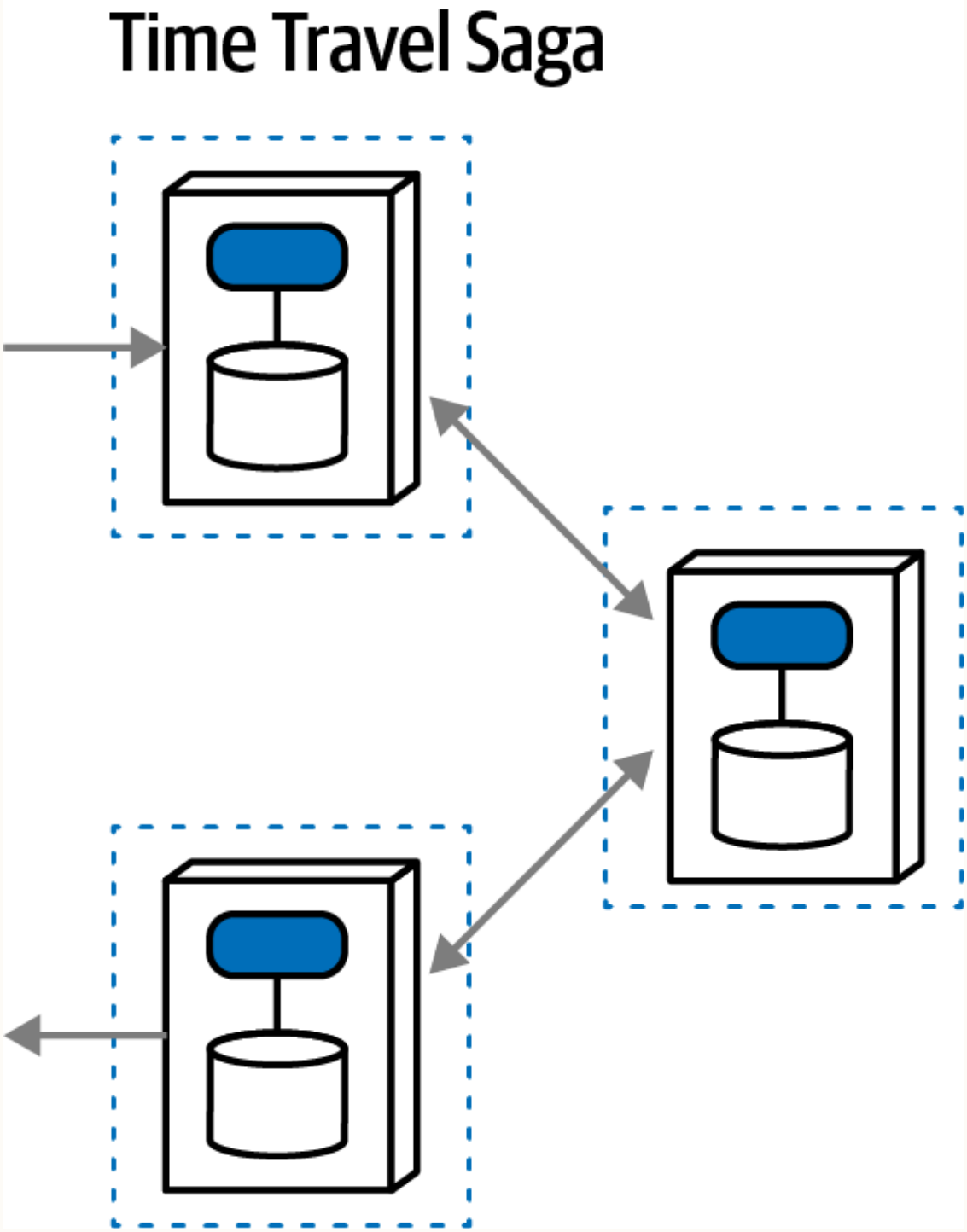


Figure 12-11. Complex workflows become difficult to manage without orchestration

In this workflow, each service accepts a request, performs an action, and then forwards the request on to another service. This architecture can implement the *Chain of Responsibility* design pattern or the *Pipes and Filters* architecture style—any workflow with a one-way series of steps. Each service in this pattern "owns" its own transactionality, so architects must design workflow error conditions into the domain design. In general, a proportional complexity relationship exists between workflow complexity and choreographed solutions because of a lack of built-in coordination via a mediator—the more complex the workflow, the more difficult choreography becomes. It is called Time Travel Saga[(sec)] because everything is decoupled from a time standpoint: each service owns its own transactional context, making workflow consistency temporally gradual—the state will become consistent over time based on the design of the interaction.

The lack of transactions in the Time Travel Saga(sec) pattern makes workflows easier to model; however, the lack of an orchestrator means that each domain service must include most workflow state and information. As in all choreographed solutions, a direct correlation exists between workflow complexity and the utility of an orchestrator; thus, this pattern is best suited for simple workflows.

For solutions that benefit from high throughput, this pattern works extremely well for "fire and forget" style workflows, such as electronic data ingestion, bulk transactions, and so on. However, because no orchestrator exists, domain services must deal with error conditions and coordination.

Lack of coupling increases scalability with this pattern; only adding asynchronicity would make it more scalable (as in the Anthology Saga(aec) pattern). However, because this pattern lacks holistic transactional coordination, architects must take extra effort to synchronize data.

Here is the qualitative evaluation of the Time Travel Saga(sec) pattern:

Coupling level

The coupling level falls in the medium range with the Time Travel Saga(sec), with the decreased coupling brought on by the absence of an orchestrator balanced by the still remaining coupling of synchronous communication. As with all eventual consistency patterns, the absence of transactional coupling eases many data concerns.

Complexity level

The loss of transactionality provides a decrease in complexity for this pattern. This pattern is quasi-special-purpose, superbly suited to fast throughput, one-way communication architectures, and the coupling level matches that style of architecture well.

Responsiveness/availability

Responsiveness scores a *medium* with this architectural pattern: it is quite high for built-to-purpose systems, as described previously, and quite low for complex error handling. Because no orchestrator exists in this pattern, each domain service must handle the scenario to restore eventual consistency in the case of an error condition, which will cause a lot of overhead with synchronous calls, impacting responsiveness and performance.

Scale/elasticity

This architecture pattern offers extremely good scale and elasticity; it could only be made better with asynchronicity (see the Anthology Saga(aec) pattern).

The ratings for the Time Travel Saga(sec) pattern appear in Table 12-5.

Table 12-5. Ratings for the *Time Travel Saga(sec)*

Time Travel Saga(sec)	Ratings
Communication	Synchronous
Consistency	Eventual
Coordination	Choreographed
Coupling	Medium
Complexity	Low
Responsiveness/availability	Medium
Scale/elasticity	High

The Time Travel Saga(sec) pattern provides an on-ramp to the more complex but ultimately scalable Anthology Saga(aec) pattern. Architects and developers find dealing with synchronous communication easier to reason about, implement, and debug; if this pattern provides adequate scalability, teams don't have to embrace the more complex but more scalable alternatives.

Fantasy Fiction Saga(aao) Pattern

The *Fantasy Fiction Saga(aao)* uses *atomic* consistency, *asynchronous* communication, and *orchestrated* coordination, as shown in Figure 12-12.

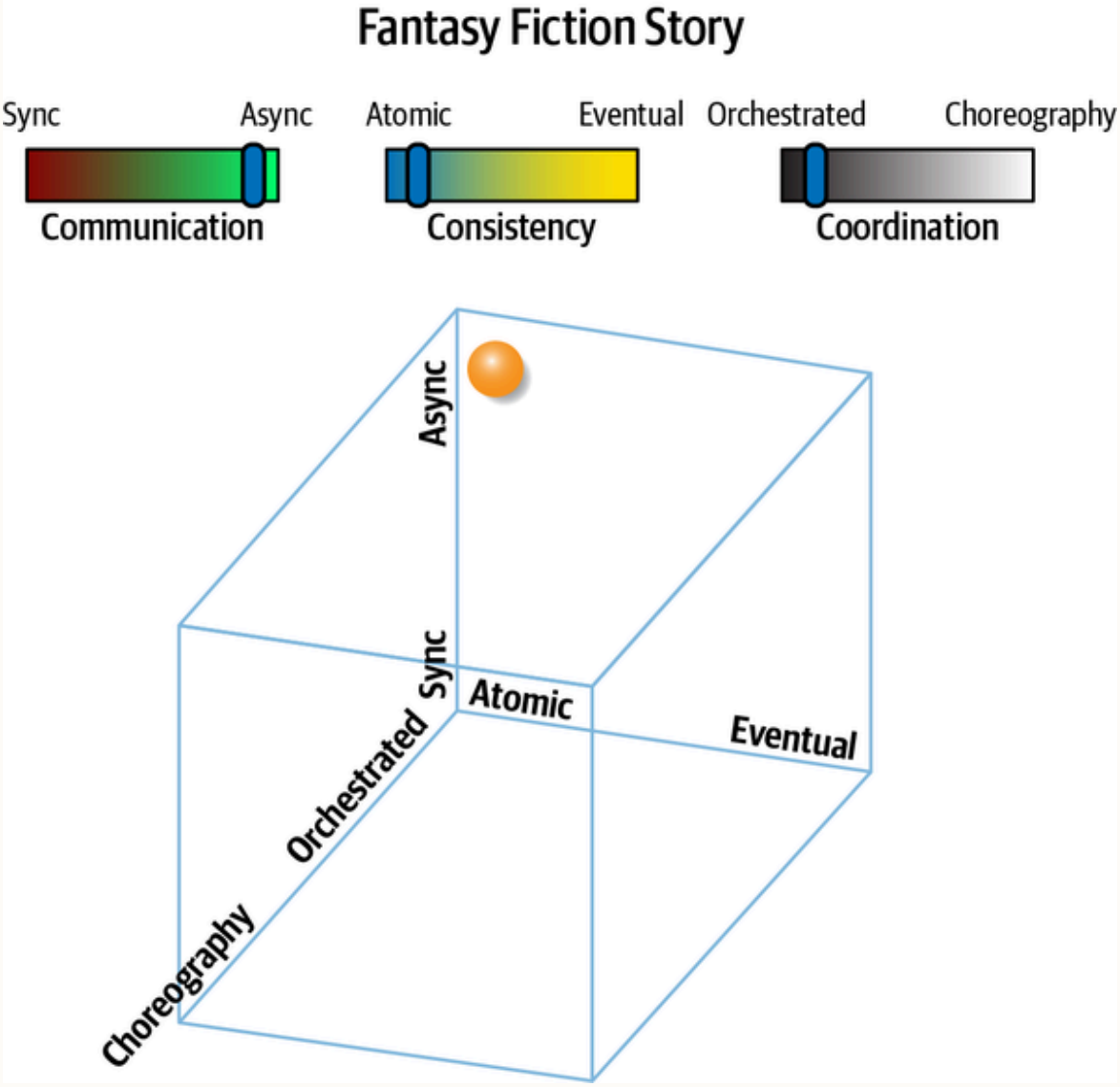


Figure 12-12. Asynchronous communication makes transactionality difficult in this pattern

The structure representation shown in Figure 12-13 starts to show some of the difficulties with this pattern.

Just because a combination of architectural forces exists doesn't mean it forms an attractive pattern, yet this relatively implausible combination has uses. This pattern resembles the Epic Saga(sao) in all aspects except for *communication*—this pattern uses *asynchronous* rather than *synchronous* communication. Traditionally, one way that architects increase the responsiveness of distributed systems is by using asynchronicity, allowing operations to occur in parallel rather than serially. This may seem like a good way to increase the perceived performance over an Epic Saga(sao).

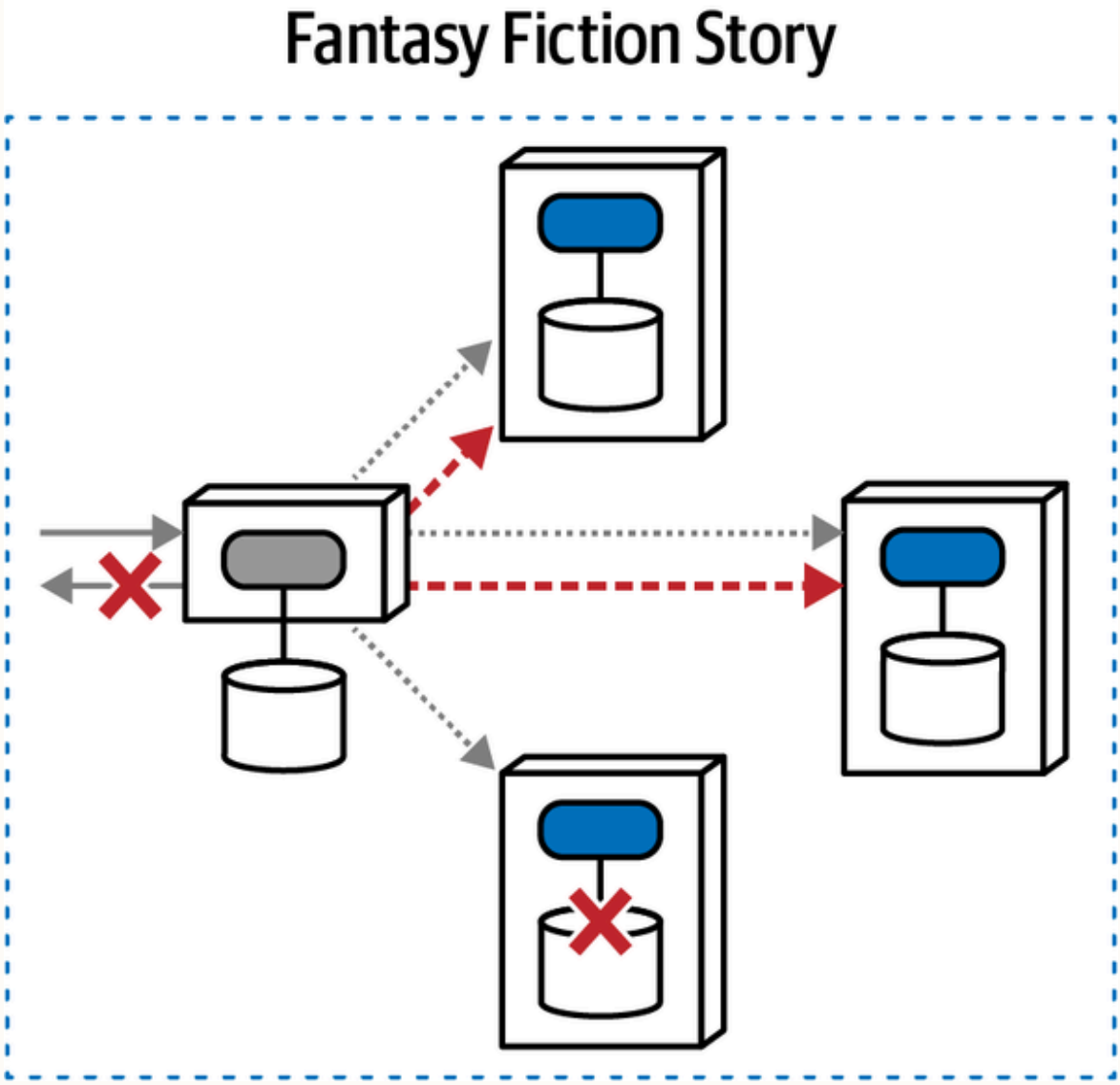


Figure 12-13. The Fantasy Fiction Saga[aao] pattern is far-fetched because transaction coordination for asynchronous communication presents difficulties

However, asynchronicity isn't a simple change—it adds many layers of complexity to architecture, especially around coordination, requiring much more complexity in the mediator. For example, suppose a transactional workflow *Alpha* begins. Because everything is asynchronous, while Alpha is pending, transactional workflow Beta begins. Now, the mediator must keep track of the state of all ongoing transactions in pending state.

It gets worse. Suppose that workflow *Gamma* begins, but the first call to the domain service depends on the still pending outcome of Alpha—how can an architect model this behavior? While possible, the complexity grows and grows.

Adding asynchronicity to orchestrated workflows adds asynchronous transactional state to the equation, removing serial assumptions about ordering and adding the possibilities of deadlocks, race conditions, and a host of other parallel system challenges.

This pattern offers the following challenges:

Coupling level

The coupling level is extremely high in this pattern, using an orchestrator and atomicity but with asynchronous communication, which makes coordination more difficult because architects and developers must deal with race conditions and other out-of-order problems imposed by asynchronous communication.

Complexity level

Because the coupling is so difficult, the complexity rises in this pattern as well. There's not only *design* complexity, requiring architects to develop overly complex workflows, but also *debugging* and *operational* complexity of dealing with asynchronous workflows at scale.

Responsiveness/availability

Because this pattern attempts transactional coordination across calls, responsiveness will be impacted overall and be extremely bad if one or more of the services isn't available.

Scale/elasticity

High scale is virtually impossible in transaction systems, even with asynchronicity. Scale is much better in the similar pattern Parallel Saga(aeo), which switches *atomic* to *eventual consistency*.

The ratings for the Fantasy Fiction Saga(aao) pattern appear in Table 12-6.

Table 12-6. Ratings for the Fantasy Fiction Saga(aao)

Fantasy Fiction	Ratings
Communication	Asynchronous
Consistency	Atomic
Coordination	Orchestrated
Coupling	High
Complexity	High
Responsiveness/availability	Low
Scale/elasticity	Low

This pattern is unfortunately more popular than it should be, mostly from the mis-guided attempt to improve the performance of Epic Saga(sao) while maintaining transactionality; a better option is usually Parallel Saga(aeo).

Horror Story(aac) Pattern

One of the patterns must be the worst possible combination; it is the aptly named *Horror Story(aac) pattern*, characterized by *asynchronous* communication, *atomic* consistency, and *choreographed* coordination, illustrated in Figure 12-14.

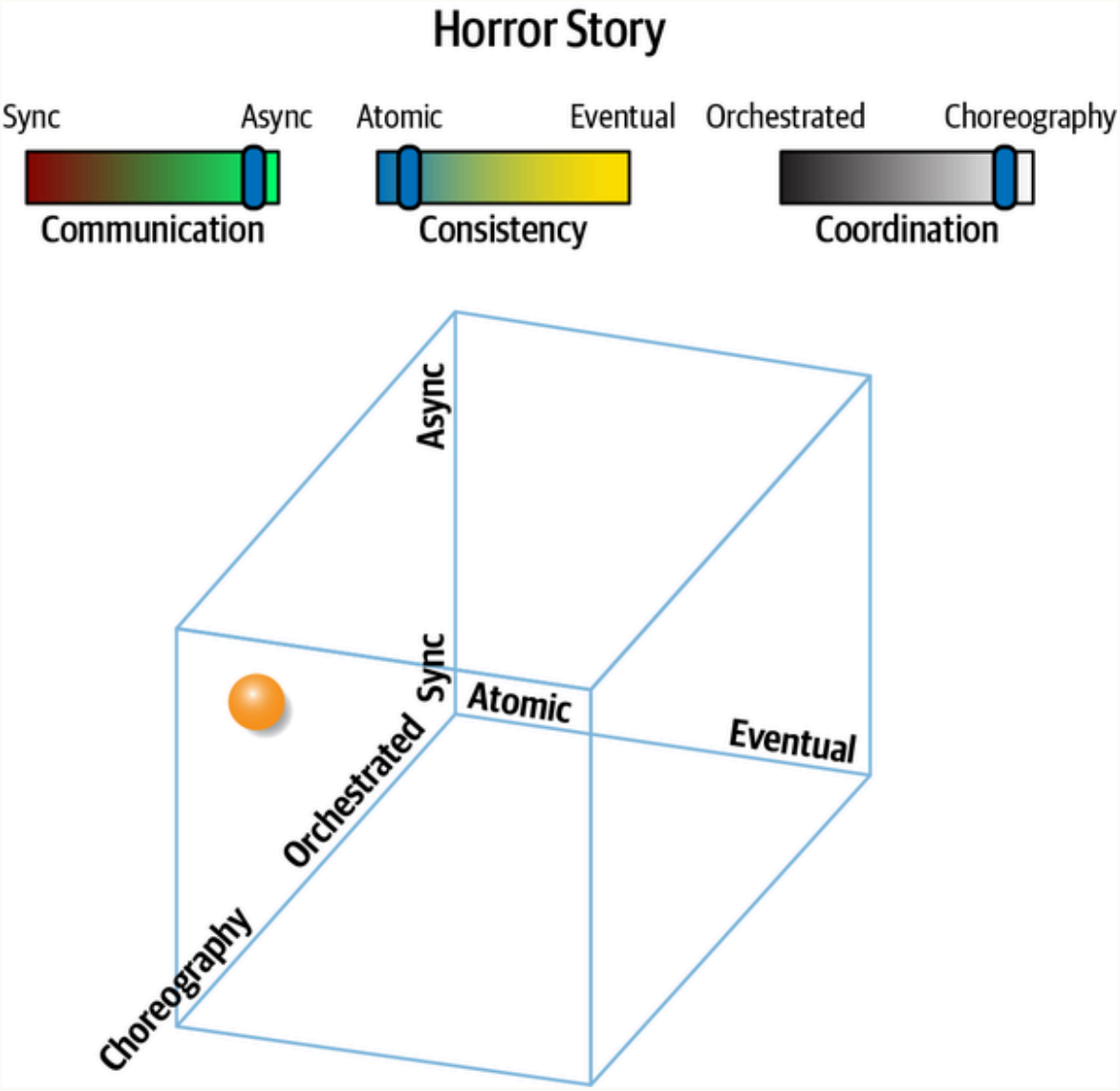


Figure 12-14. The most difficult combination: achieving transactionality while asynchronous and choreographed

Why is this combination so horrible? It combines the most stringent coupling around consistency (*atomic*) with the two loosest coupling styles, *asynchronous* and *choreography*. The structural communication for this pattern appears in Figure 12-15.

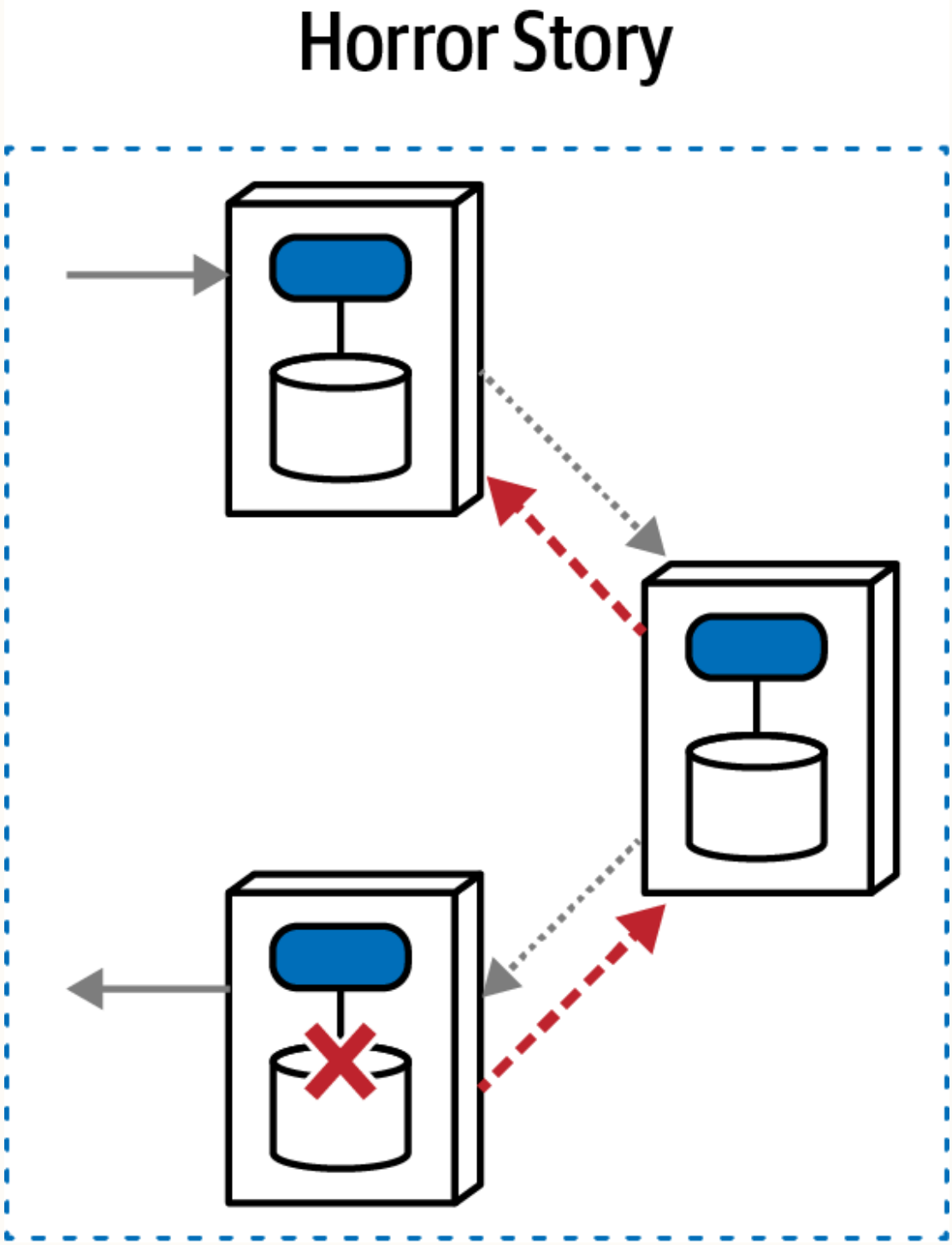


Figure 12-15. This pattern requires a lot of interservice communication because of required transactionality and the lack of a mediator

In this pattern, no mediator exists to manage transactional consistency across multiple services—while using asynchronous communication. Thus, each domain service must track undo information about multiple pending transactions, potentially out of order because of asynchronicity, and coordinate with each other during error conditions. For just one of many possible horrible examples, imagine that transaction *Alpha* starts and, while pending, transaction *Beta* starts. One of the calls for the *Alpha* transaction fails—now, the choreographed services have to reverse the order of firing, undoing each (potentially out-of-order) element of the transaction along the way. The multiplicity and complexity of error conditions makes this a daunting option.

Why might an architect choose this option? Asynchronicity is appealing as a performance boost, yet the architect may still try to maintain transactional integrity, which has many

myriad failure modes. Instead, an architect would be better off choosing the Anthology Saga(aec) pattern, which removes holistic transactionality.

The qualitative evaluations for the Horror Story(aac) pattern are as follows:

Coupling level

Surprisingly, the coupling level for this pattern isn't the worst (that "honor" goes to the Epic Saga(sao) pattern). While this pattern does attempt the worst kind of single coupling (transactionality), it relieves the other two, lacking both a mediator and the coupling—increasing synchronous communication.

Complexity level

Just as the name implies, the complexity of this pattern is truly horrific, the worst of any because it requires the most stringent requirement (transactionality) with the most difficult combination of other factors to achieve that (asynchronicity and choreography).

Scale/elasticity

This pattern does scale better than ones with a mediator, and asynchronicity also adds the ability to perform more work in parallel.

Responsiveness/availability

Responsiveness is *low* for this pattern, similar to the other patterns that require holistic transactions: coordination for the workflow requires a large amount of inter-service "chatter," hurting performance and responsiveness.

The trade-offs for the Horror Story(aac) pattern appear in Table 12-7.

Table 12-7. Ratings for the Horror Story(aac)

Horror Story(aac)	Ratings
Communication	Asynchronous
Consistency	Atomic
Coordination	Choreographed
Coupling	Medium
Complexity	Very high
Responsiveness/availability	Low
Scale/elasticity	Medium

The aptly named Horror Story(aac) pattern is often the result of a well-meaning architect starting with an Epic Saga(sao) pattern, noticing slow performance because of complex workflows, and realizing that techniques to improve performance include asynchronous communication and choreography. However, this thinking provides an excellent example of not considering all the entangled dimensions of a problem space. In isolation, asynchronous communication improves performance. However, as architects, we cannot consider it in isolation when it is entangled with other architecture dimensions, such as consistency and coordination.

Parallel Saga(aeo) Pattern

The *Parallel Saga(aeo)* pattern is named after the "traditional" Epic Saga(sao) pattern with two key differences that ease restrictions and therefore make it an easier pattern to implement: asynchronous communication and eventual consistency. The dimensional diagram of the Parallel Saga(aeo) pattern appears in Figure 12-16.

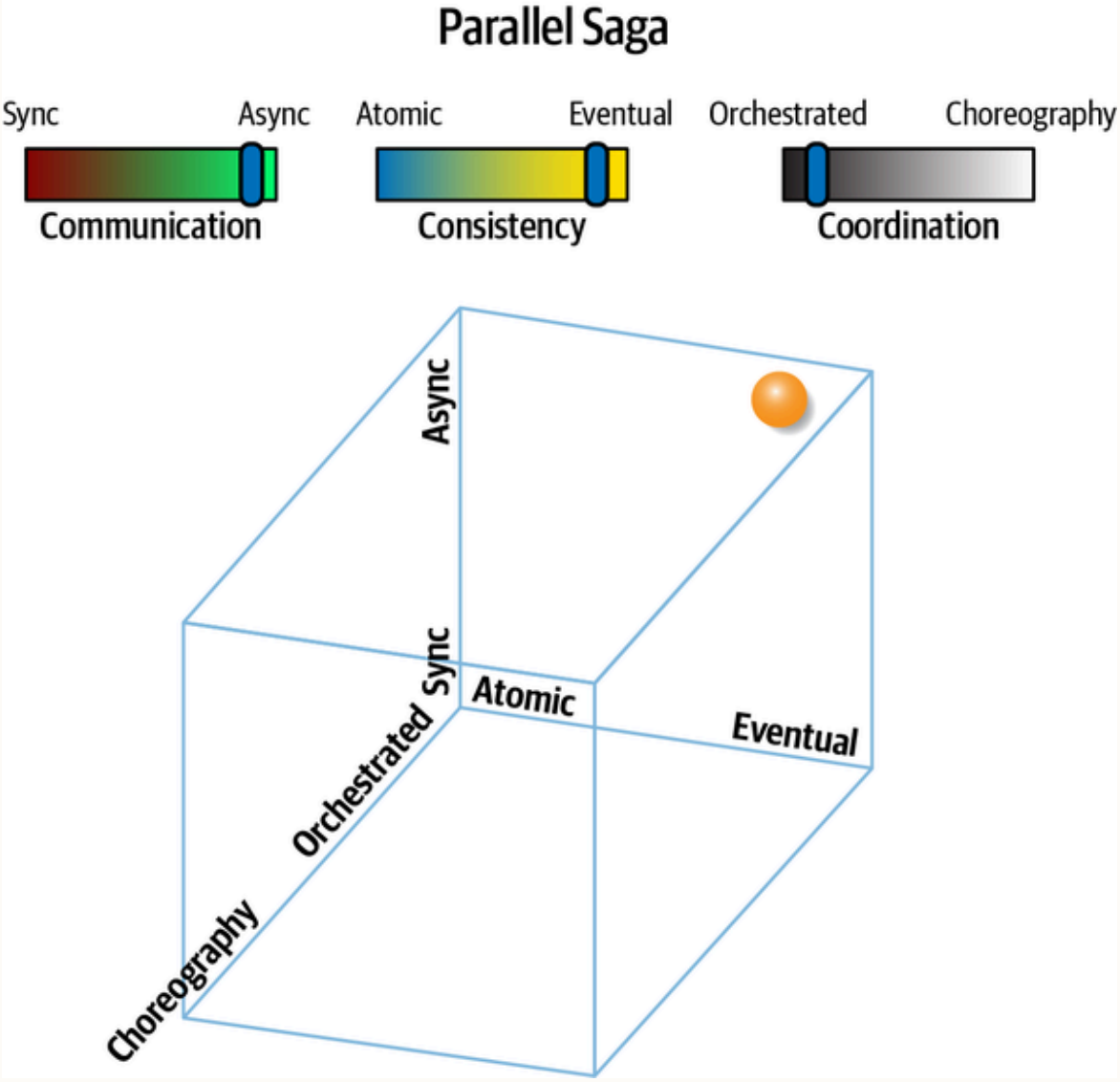


Figure 12-16. Parallel Saga(aeo) offers performance improvements over traditional sagas

The most difficult goals in the Epic Saga(sao) pattern revolve around transactions and synchronous communication, both of which cause bottlenecks and performance degradation. As shown in Figure 12-16, the pattern loosens both restraints.

The isomorphic representation of Parallel Saga(aeo) appears in Figure 12-17.

This pattern uses a mediator, making it suitable for complex workflows. However, it uses asynchronous communication, allowing for better responsiveness and parallel execution. Consistency in the pattern lies with the domain services, which may require some synchronization of shared data, either in the background or driven via the mediator. As in other architectural problems that require coordination, a mediator becomes quite useful.

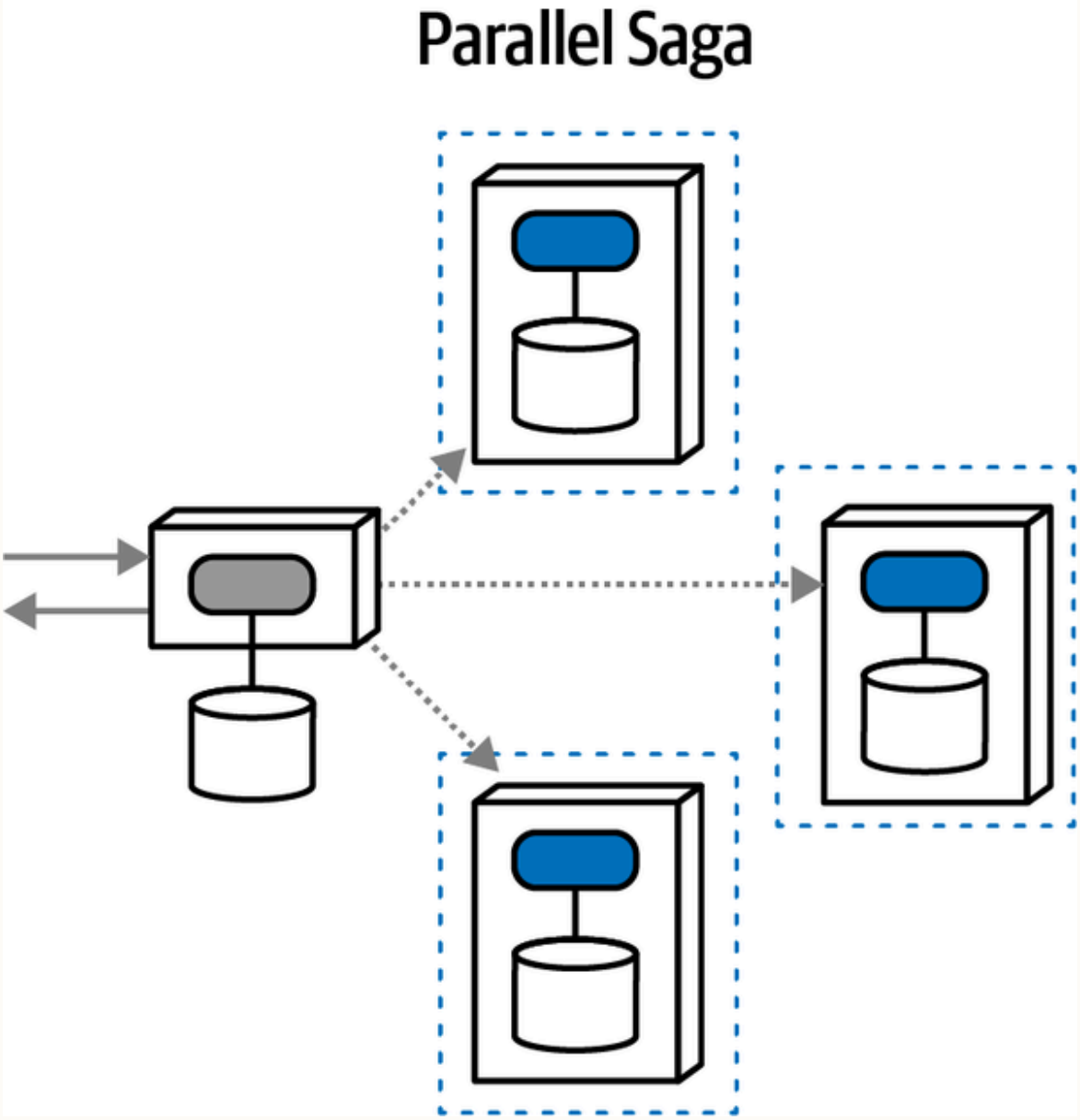


Figure 12-17. Each service owns its own transactionality; the mediator coordinates request and response

For example, if an error occurs during the execution of a workflow, the mediator can send asynchronous messages to each involved domain service to compensate for the failed change, which may entail retries, data synchronization, or a host of other remediations.

Of course, the loosening of constraints implies that some benefits will be traded off, which is the nature of software architecture. Lack of transactionality imposes more burden on the mediator to resolve error and other workflow issues. Asynchronous communication, while offering better responsiveness, makes resolving timing and synchronization issues difficult—race conditions, deadlocks, queue reliability, and a host of other distributed architecture headaches reside in this space.

The Parallel Saga[(aeo)] pattern exhibits the following qualitative scores:

Coupling level

This pattern has a low coupling level, isolating the coupling-intensifying force of transactions to the scope of the individual domain services. It also utilizes asynchro-

nous communication, further decoupling services from wait states, allowing for more parallel processing but adding a time element to an architect's coupling analysis.

Complexity level

The complexity of the Parallel Saga(aeo) is also low, reflecting the lessening of coupling stated previously. This pattern is fairly easy for architects to understand, and orchestration allows for simpler workflow and error-handling designs.

Scale/elasticity

Using asynchronous communication and smaller transaction boundaries allows this architecture to scale quite nicely, and with good levels of isolation between services. For example, in a microservices architecture, some public-facing services might need higher levels of scale and elasticity, where back office services don't need scale but higher levels of security. Isolating transactions at the domain level frees the architecture to scale around domain concepts.

Responsiveness/availability

Because of lack of coordinated transactions and asynchronous communication, the responsiveness of this architecture is *high*. In fact, because each of these services maintains its own transactional context, this architecture is well suited to highly variable service performance footprints between services, allowing architects to scale some services more than others because of demand.

The ratings associated with the Parallel Saga(aeo) pattern appear in Table 12-8.

Table 12-8. Ratings for the Parallel Saga(aeo)

Parallel Saga(aeo)	Ratings
Communication	Asynchronous
Consistency	Eventual
Coordination	Orchestrated
Coupling	Low
Complexity	Low
Responsiveness/availability	High
Scale/elasticity	High

Overall, the Parallel Saga(aeo) pattern offers an attractive set of trade-offs for many scenarios, especially with complex workflows that need high scale.

Anthology Saga(aec) Pattern

The *Anthology Saga(aec) pattern* provides the exact opposite set of characteristics to the traditional Epic Saga(sao) pattern: it utilizes *asynchronous* communication, *eventual* consistency, and *choreographed* coordination, providing the least coupled exemplar among all these patterns. The dimensional view of the Anthology Saga(aec) pattern appears in Figure 12-18.

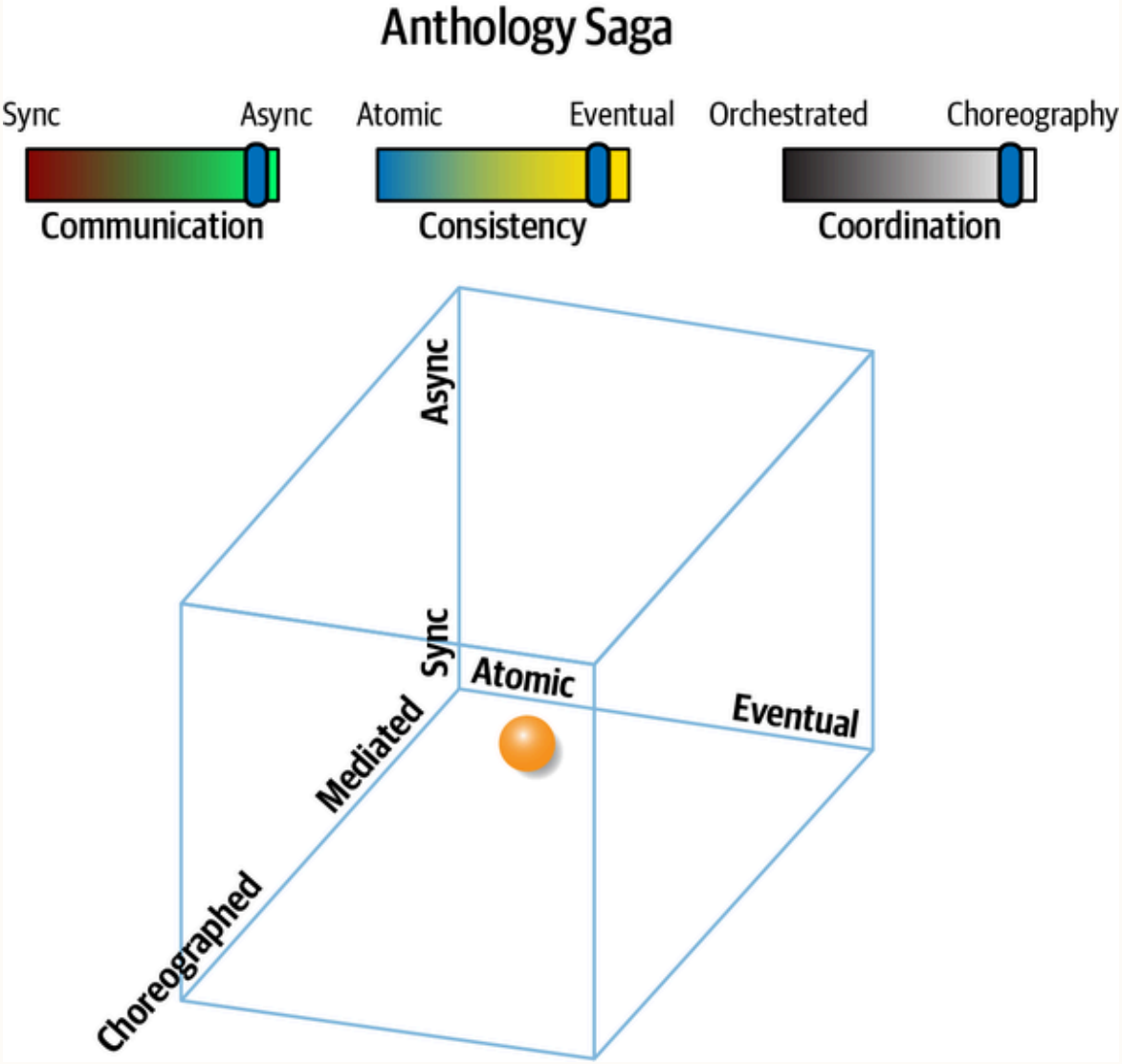


Figure 12-18. The Anthology Saga(aec) pattern offers the opposite extremes of the Epic Saga, and is therefore the least coupled pattern

The anthology pattern uses message queues to send asynchronous messages to other domain services without orchestration, as illustrated in Figure 12-19.

As you can see, each service maintains its own transactional integrity, and no orchestrator exists, forcing each domain service to include more context about the workflows they participate in, including error handling and other coordination strategies.

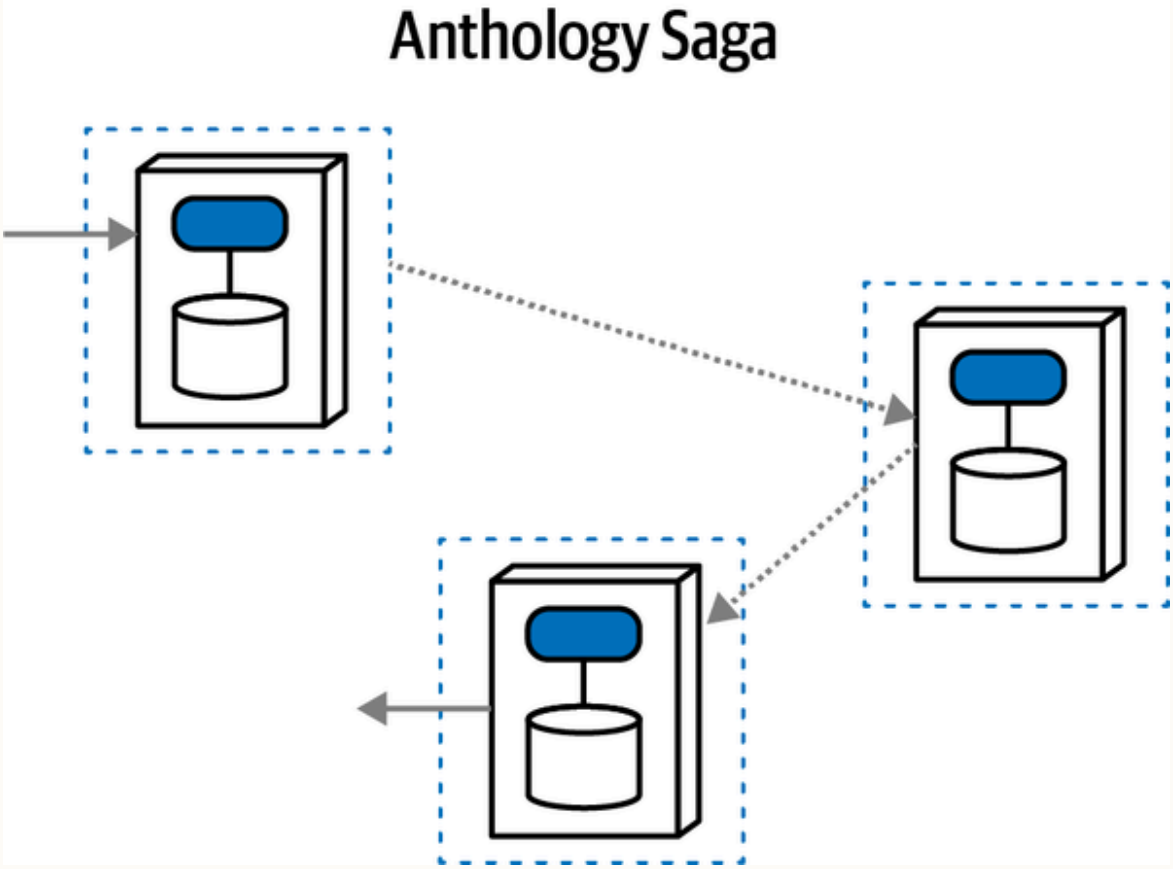


Figure 12-19. Lack of orchestration, eventual consistency, and asynchronicity make this pattern highly decoupled but a challenge for coordination

The lack of orchestration makes services more complex but allows for much higher throughput, scalability, elasticity, and other beneficial operational architecture characteristics. No bottlenecks or coupling choke points exist in this architecture, allowing for high responsiveness and scalability.

However, this pattern doesn't work particularly well for complex workflows, especially around resolving data consistency errors. While it may not seem possible without an orchestrator, stamp coupling ("Stamp Coupling for Workflow Management") may be used to carry workflow state, as described in the similar Phone Tag Saga(sac) pattern.

This pattern works best for simple, mostly linear workflows, where architects desire high processing throughput. This pattern provides the most potential for both high performance and scale, making it an attractive choice when those are key drivers for the system. However, the degree of decoupling makes coordination difficult, prohibitively so for complex or critical workflows.

The short-story-inspired Anthology Saga(aec) pattern has the following characteristics:

Coupling level

Coupling for this pattern is the lowest for any other combination of forces, creating a highly decoupled architecture well suited for high scale and elasticity.

Complexity level

While the coupling is extremely low, complexity is correspondingly high, especially for complex workflows where an orchestrator (lacking here) is convenient.

Scale/elasticity

This pattern scores the highest in the scale and elasticity category, correlating with the overall lack of coupling found in this pattern.

Responsiveness

Responsiveness is high in this architecture because of a lack of speed governors (transactional consistency, synchronous communication) and use of responsiveness accelerators (choreographed coordination).

The ratings table for the Anthology Saga(aec) pattern appear in Table 12-9.

Table 12-9. Ratings for the Anthology Saga(aec)

Anthology Saga(aec)	**Ratings**
Communication	Asynchronous
Consistency	Eventual
Coordination	Choreographed
Coupling	Very low
Complexity	High
Responsiveness/availability	High
Scale/elasticity	Very high

The Anthology Saga(aec) pattern is well suited to extremely high throughput communication with simple or infrequent error conditions. For example, a *Pipes and Filters* architecture would fit this pattern exactly.

Architects can implement the patterns described in this section in a variety of ways. For example, architects can manage transactional sagas through atomic transactions by using compensating updates or by managing transactional state with eventual consistency. This section showed the advantages and disadvantages of each approach, which will help an architect decide which transactional saga pattern to use.

State Management and Eventual Consistency

State management and eventual consistency leverage *finite state machines* (see "Saga State Machines") to always know the current state of the transactional saga, and to also eventually correct the error condition through retries or some sort of automated or manual corrective action. To illustrate this approach, consider the Fairy Tale Saga(seo) implementation of the ticket completion example illustrated in Figure 12-20.

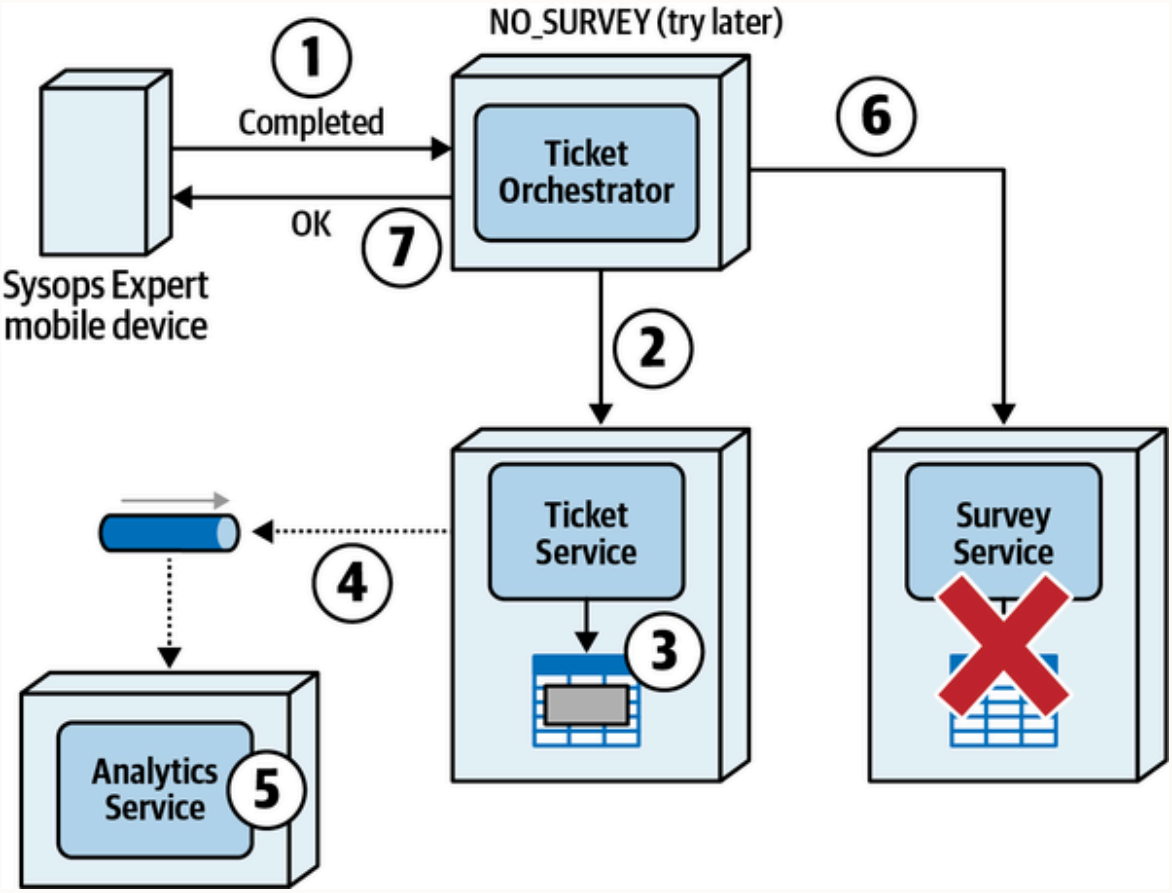


Figure 12-20. The Fairy Tale Saga leads to better responsiveness, but leaves data sources out of sync with one another until they can be corrected

Notice that the Survey Service is not available during the scope of the distributed transaction. However, with this type of saga, rather than issue a compensating update, the state of the saga is changed to `NO_SURVEY` and a successful response is sent to the Sysops Expert (step 7 in the diagram). The Ticket Orchestrator Service then works asynchronously (behind the scenes) to resolve the error programmatically by retries and error analysis. If it cannot resolve the error, the Ticket Orchestrator Service sends the error to an administrator or supervisor for manual repair and processing.

By managing the *state* of the saga rather than issuing compensating updates, the end user (in this case, the Sysops Squad expert) doesn't need to be concerned that the survey was not sent to the customer—that responsibility is for the Ticket Orchestrator Service to worry about. Responsiveness is good from the end user's perspective, and the user can work on other tasks while the errors are handled by the system.

Saga State Machines

A *state machine* is a pattern that describes all of the possible paths that can exist within a distributed architecture. A state machine always starts with a beginning state that launches the transactional saga, then contains transition states and corresponding action that should occur when the transition state happens.

To illustrate how a saga state machine works, consider the following workflow of a new problem ticket created by a customer in the Sysops Squad system:

1. The customer enters a new problem ticket into the system.
2. The ticket is assigned to the next available Sysops Squad expert.
3. The ticket is then routed to the expert's mobile device.
4. The expert receives the ticket and works on the issue.
5. The expert finishes the repair and marks the ticket as complete.
6. A survey is sent to the customer.

The various states that can exist within this transactional saga, as well as the corresponding transition actions, are illustrated in Figure 12-21. Notice that the transactional saga begins with the `START` node indicating the saga entry point, and terminates with the `CLOSED` node indicating the saga exit point.

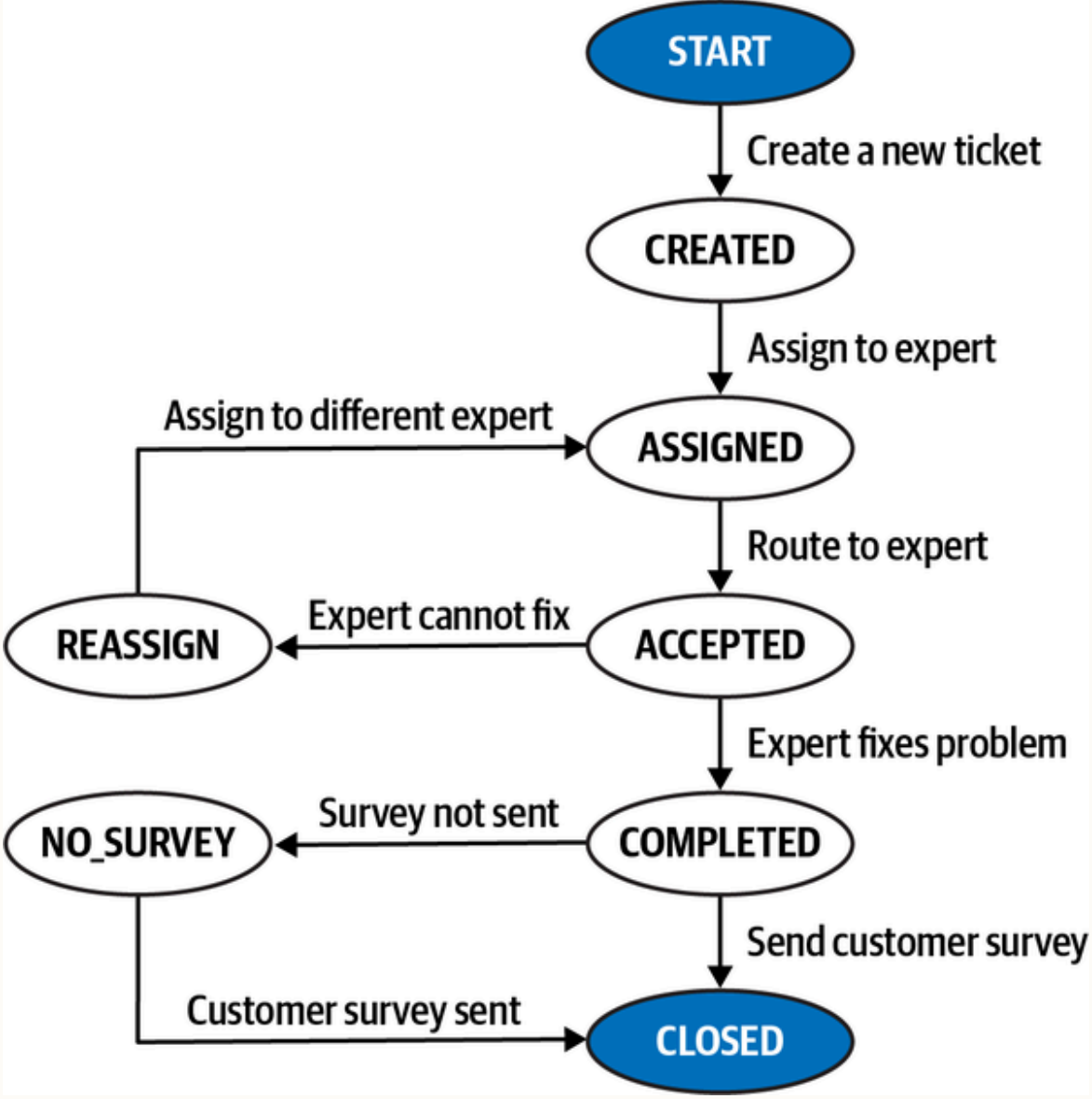


Figure 12-21. State diagram for creating a new problem ticket

The following items describe in more detail this transactional saga and the corresponding states and transition actions that happen within each state:

START

The transactional saga starts with a customer entering a new problem ticket into the system. The customer's support plan is verified, and the ticket data is validated. Once the ticket is inserted into the ticket table in the database, the transactional saga state moves to CREATED and the customer is notified that the ticket has been successfully created. This is the only possible outcome for this state transition—any errors within this state prevent the saga from starting.

CREATED

Once the ticket is successfully created, it is assigned to a Sysops Squad expert. If no expert is available to service the ticket, it is held in a wait state until an expert is available. Once an expert is assigned, the saga state moves to the ASSIGNED state. This is the only outcome for this state transition, meaning the ticket is held in CREATED state until it can be assigned.

ASSIGNED

Once a ticket is assigned to an expert, the only possible outcome is to route the ticket to the expert. It is assumed that during the assignment algorithm, the expert has been located and is available. If the ticket cannot be routed because the expert cannot be located or is unavailable, the saga stays in this state until it can be routed. Once routed, the expert must acknowledge that the ticket has been received. Once this happens, the transactional saga state moves to ACCEPTED. This is the only possible outcome for this state transition.

ACCEPTED

There are two possible states once a ticket has been accepted by a Sysops Squad expert: COMPLETED or REASSIGN. Once the expert finishes the repair and marks the ticket as "complete," the state of the saga moves to COMPLETED. However, if for some reason the ticket was wrongly assigned or the expert is not able to finish the repair, the expert notifies the system and the state moves to REASSIGN.

REASSIGN

Once in this saga state, the system will reassign the ticket to a different expert. Like the CREATED state, if an expert is not available, the transactional saga will remain in the REASSIGN state until an expert is assigned. Once a different expert is found and the ticket is once again assigned, the state moves into the ASSIGNED state, waiting to be accepted by the other expert. This is the only possible outcome for this state transition, and the saga remains in this state until an expert is assigned to the ticket.

COMPLETED

The two possible states once an expert completes a ticket are CLOSED or NO_SURVEY. When the ticket is in this state, a survey is sent to the customer to rate the expert and the service, and the saga state is moved to CLOSED, thus ending the transaction saga. However, if the Survey Service is unavailable or an error occurs while sending the survey, the state moves to NO_SURVEY, indicating that the issue was fixed but no survey was sent to the customer.

NO_SURVEY

In this error condition state, the system continues to try sending the survey to the customer. Once successfully sent, the state moves to CLOSED, marking the end of the transactional saga. This is the only possible outcome of this state transaction.

In many cases, it's useful to put the list of all possible state transitions and the corresponding transition action in some sort of table. Developers can then use this table to implement the state transition triggers and possible error conditions in an orchestration service (or re-

spective services if using choreography). An example of this practice is shown in Table 12-10, which lists all the possible states and actions that are triggered when the state transition occurs.

Table 12-10. Saga state machine for a new problem ticket in the Sysops Squad system

Initiating state	Transition state	Transaction action
START	CREATED	Assign ticket to expert
CREATED	ASSIGNED	Route ticket to assigned expert
ASSIGNED	ACCEPTED	Expert fixes problem
ACCEPTED	COMPLETED	Send customer survey
ACCEPTED	REASSIGN	Reassign to a different expert
REASSIGN	ASSIGNED	Route ticket to assigned expert
COMPLETED	CLOSED	Ticket saga done
COMPLETED	NO_SURVEY	Send customer survey
NO_SURVEY	CLOSED	Ticket saga done

The choice between using compensating updates or state management for distributed transaction workflows depends on the situation as well as trade-off analysis between responsiveness and consistency. Regardless of the technique used to manage errors within a distributed transaction, the state of the distributed transaction should be known and also managed.

Table 12-11 summarizes the trade-offs associated with using state management rather than atomic distributed transactions with compensating updates.

TRADE-OFFS

Table 12-11. Trade-offs associated with state management rather than atomic distributed transactions with compensating updates

Advantages	Disadvantages
Good responsiveness	Data may be out of sync when errors occur
Less impact to end user for errors	Eventual consistency may take some time

Techniques for Managing Sagas

Distributed transactions are not something that can be simply "dropped into" a system. They cannot be downloaded or purchased using some sort of framework or product like ACID transaction managers—they must be designed, coded, and maintained by developers and architects.

One of the techniques we like to use to help manage distributed transactions is to leverage annotations (Java) or custom attributes (C#), or other similar artifacts in other languages. While these language artifacts themselves don't contain any actual functionality, they do provide a programmatic way of capturing and documenting the transactional sagas in the system, as well as provide a means for associating services with transactional sagas.

The source listings in Example 12-1 (Java) and Example 12-2 (C#) show an example of implementing these annotations and custom attributes. Notice that in both implementations, the transactional sagas (`NEW_TICKET` , `CANCEL_TICKET` , and so on) are contained within the `Transaction` enum, providing a single place within the source code for listing and documenting the various sagas that exist within an application context.

Example 12-1. Source code defining a transactional saga annotation (Java)

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.TYPE)
public @interface Saga {
  public Transaction[] value();

  public enum Transaction {
    NEW_TICKET,
    CANCEL_TICKET,
```

```
    NEW_CUSTOMER,
    UNSUBSCRIBE,
    NEW_SUPPORT_CONTRACT
  }
}
```

Example 12-2. Source code defining a transactional saga attribute (C#)

```
[AttributeUsage(AttributeTargets.Class)]
class Saga : System.Attribute {
  public Transaction[] transaction;

  public enum Transaction {
    NEW_TICKET,
    CANCEL_TICKET,
    NEW_CUSTOMER,
    UNSUBSCRIBE,
    NEW_SUPPORT_CONTRACT
  };
}
```

Once defined, these annotations or attributes can be used to identify services that are involved in the transactional saga. For example, the source code listing in Example 12-3 shows that the Survey Service (identified by the `SurveyServiceAPI` class as the service entry point) is involved in the `NEW_TICKET` saga, whereas the Ticket Service (identified by the `TicketServiceAPI` class as the service entry point) is involved in two sagas: the `NEW_TICKET` and the `CANCEL_TICKET`.

Example 12-3. Source code showing the use of the transactional saga annotation (Java)

```
@ServiceEntrypoint
@Saga(Transaction.NEW_TICKET)
public class SurveyServiceAPI {
  ...
}

@ServiceEntrypoint
@Saga({Transaction.NEW_TICKET,)
    Transaction.CANCEL_TICKET})
public class TicketServiceAPI {
  ...
}
```

Notice how the `NEW_TICKET` saga includes the Survey Service and the Ticket Service. This is valuable information to a developer because it helps them define the testing scope when making changes to a particular workflow or saga, and also lets them know what other services might be impacted by a change to one of the services within the transactional saga.

Using these annotations and custom attributes, architects and developers can write simple command-line interface (CLI) tools to walk through a codebase or source code repository to provide saga information in real time. For example, using a simple custom code-walk tool, a developer, architect, or even a business analyst can query what services are involved for the `NEW_TICKET` saga:

```
$ ./sagatool.sh NEW_TICKET -services

-> Ticket Service
-> Assignment Service
-> Routing Service
-> Survey Service

$
```

A custom code-walking tool can look at each class file in the application context containing the `@ServiceEntrypoint` custom annotation (or attribute) and check the `@Saga` custom annotation for the presence of the particular saga (in this case, `Transaction.NEW_TICKET`). This sort of custom tool is not complicated to write, and can help provide valuable information when managing transactional sagas.

Sysops Squad Saga: Atomic Transactions and Compensating Updates

`Tuesday, April 5, 09:44`



Addison and Austen met first thing with Logan to hash out the issues around transactionality in the new microservices architecture in the longish conference room.

Logan began, “I know that not everyone is on the same page about how what you’ve read applies to what we’re doing here. So, I’ve prepared some workflows and diagrams to help everyone get on the same page. Today, we’re discussing marking a ticket complete in the system. For this workflow, the Sysops Squad expert completes a job and marks the ticket as “complete” using the mobile application on the expert’s mobile device. I want to

talk about the Epic Saga pattern and the issues around compensating updates. I've created a diagram to illustrate this workflow in Figure 12-22 Can everyone see it?"

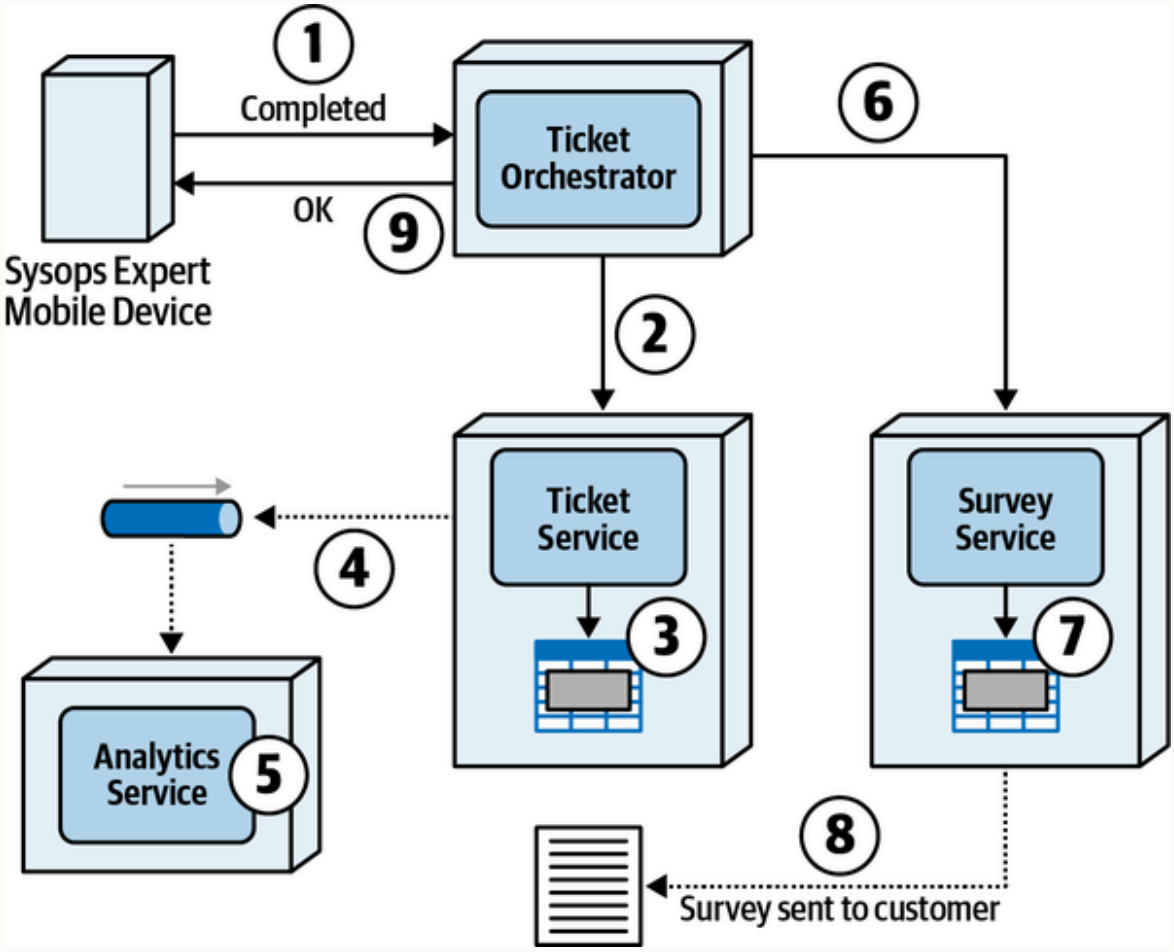


Figure 12-22. The epic saga requires the ticket status to be updated and survey to be sent in one synchronous atomic operation

Logan continued, "I've also created a list that describes each step. The circled numbers on the diagram match up with the workflow."

1. The Sysops Squad expert marks the ticket as complete using an app on their mobile device, which is synchronously received by the Ticket Orchestrator Service.
2. The Ticket Orchestrator Service sends a synchronous request to the Ticket Service to change the state of the ticket from "in-progress" to "complete."
3. The Ticket Service updates the ticket number to "complete" in the database table and commits the update.
4. As part of the ticket completion process, the Ticket Service asynchronously sends ticketing information (such as ticket repair time, ticket wait time, duration, and so on) to a queue to be picked up by the Analytics Service. Once sent, the Ticket Service sends an acknowledgment to the Ticket Orchestrator Service that the update is complete.
5. At about the same time, the Analytics Service asynchronously receives the updated ticket analytics and starts to process the ticket information.
6. The Ticket Orchestrator Service then sends a synchronous request to the Survey Service to prepare and send the customer survey to the customer.

7. The Survey Service inserts data into a table with the survey information (customer, ticket info, and timestamp) and commits the insert.
8. The Survey Service then sends the survey to the customer via email and returns an acknowledgment back to the Ticket Orchestrator Service that the survey processing is complete.
9. Finally, the Ticket Orchestrator Service sends a response back to the Sysops Squad expert's mobile device stating that the ticket completion processing is done. Once this happens, the expert can select the next problem ticket assigned to them.

"Wow, this is really helpful. How long did it take you to create this?" said Addison.

"Not a little time, but it's come in handy. You aren't the only group that's confused about how to get all these moving pieces to work together. This is the hard part of software architecture. Everyone understand the basics of the workflow?"

To a sea of nods, Logan continued, "One of the first issues that occurs with compensating updates is that since there's no transactional isolation within a distributed transaction (see "Distributed Transactions"), other services may have taken action on the data updated within the scope of the distributed transaction before the distributed transaction is complete. To illustrate this issue, consider the same Epic Saga example appearing in Figure 12-23: the Sysops Squad expert marks a ticket as complete, but this time the Survey Service is not available. In this case, a compensating update (step 7 in the diagram) is sent to the Ticket Service to reverse the update, changing the ticket state from *completed* back to *in-progress* (step 8 in the diagram)."

"Notice also in Figure 12-23 that since this is an atomic distributed transaction, an error is then sent back to the Sysops Squad expert indicating that the action was not successful and to try again. Now, a question for you: why should the Sysops Squad expert have to worry that the survey is not sent?"

Austen pondered a moment. "But wasn't that part of the workflow in the monolith? All that stuff happened within a transaction, if I remember correctly."

"Yeah, but I always thought that was weird, just never said anything," said Addison. "I don't see why the expert should worry about the survey. The expert just wants to get on to the next ticket assigned to them."

"Right," Logan said. "This is the issue with atomic distributed transactions—the end user is unnecessarily semantically coupled to the business process. But notice that Figure 12-23 also illustrates the issue with the lack of transaction isolation within a distributed transaction. Notice that as part of the original update to mark the ticket as *complete*, the Ticket Service asynchronously sent the ticket information to a queue (step 4 in the diagram) to be

processed by the Analytics Service (step 5). However, when the compensating update is issued to the Ticket Service (step 7), the ticket information has already been processed by the Analytics Service in step 5."

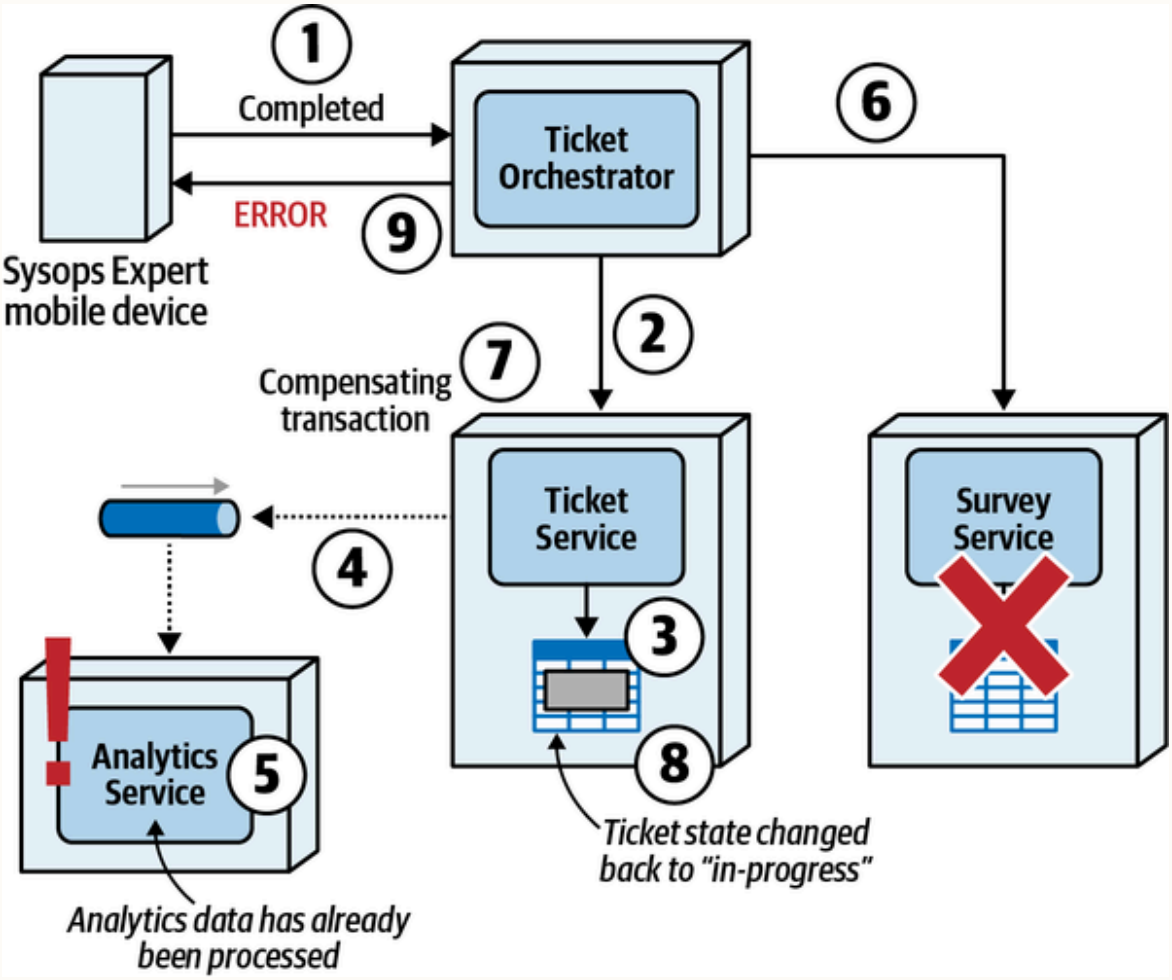


Figure 12-23. Epic Saga(sao) requires compensation, but side effects can occur

"We call this a *side effect* within distributed architectures. By reversing the transaction in the Ticket Service, actions performed by other services using data from the prior update may have already taken place and might not be able to be reversed. This scenario points to the importance of *isolation* within a transaction, something that distributed transactions do not support. To address this issue, the Ticket Service could send another request through the data pump to the Analytics Service, telling that service to ignore the prior ticket information, but just imagine the amount of complex code and timing logic that would be required in the Analytics Service to address this compensating change. Furthermore, there may have been additional downstream actions taken on the analytical data already processed by the Analytics Service, further complicating the chain of events to reverse and correct. With distributed architectures and distributed transactions, it really is sometimes *turtles all the way down*."

Logan paused for a moment, then continued, "Another issue—"

Austen interrupted, "Another issue?"

Logan smiled. "Another issue regarding compensating updates is compensation failures. Keeping with the same Epic Saga example for completing a ticket, notice in Figure 12-24 that in step 7 a compensating update is issued to the Ticket Service to change the state from *completed* back to *in-progress.* However, in this case, the Ticket Service generates an error when trying to change the state of the ticket (step 8)."

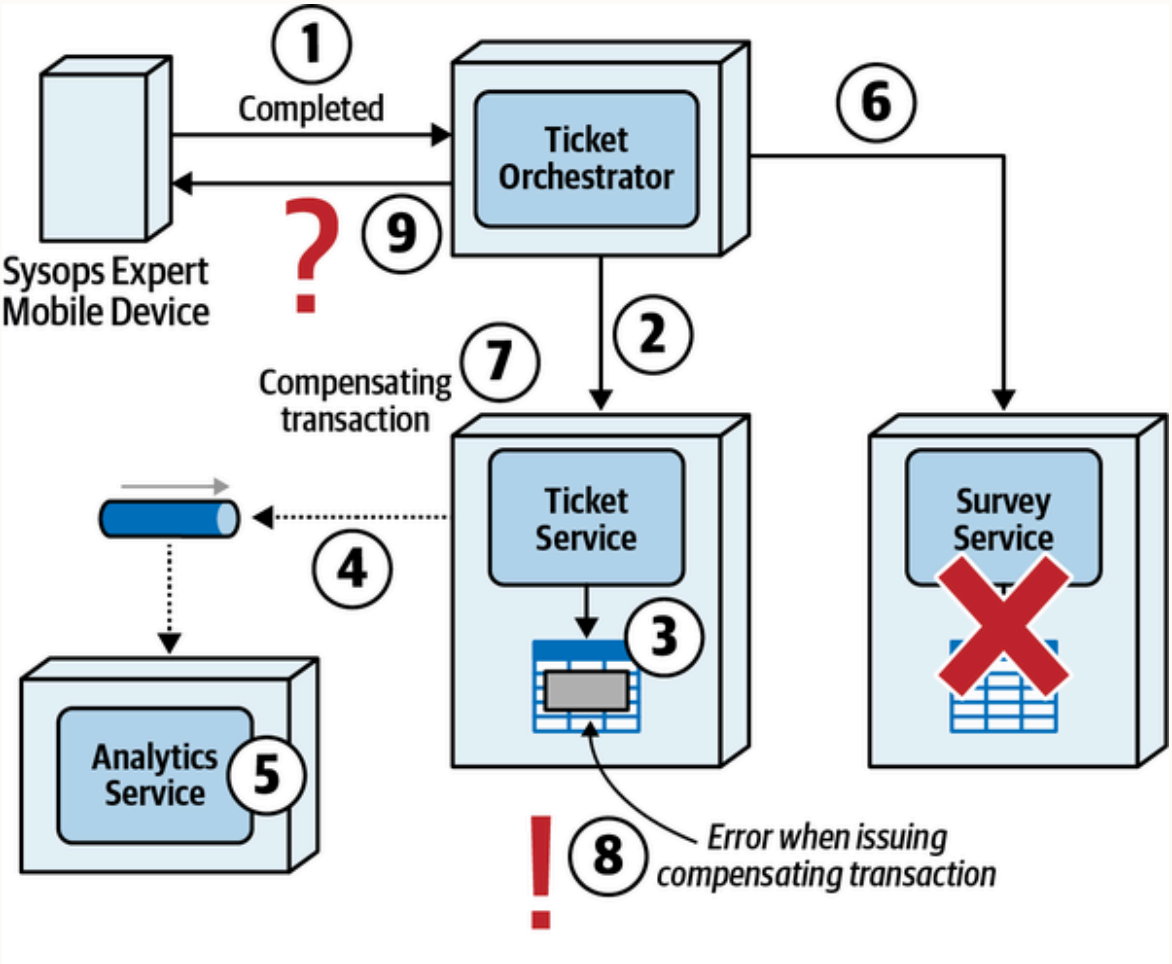


Figure 12-24. Compensating updates within an Epic Saga can fail, leading to inconsistency and confusion about what action to take in the event of a compensation failure

"I've seen that happen! It took forever to track that down," said Addison.

"Architects and developers tend to assume that compensating updates will always work," Logan said. "But sometimes they don't. In this case, as shown in Figure 12-24, there is confusion about what sort of response to send back to the end user (in this case, the Sysops Squad expert). The ticket status is already marked as *complete* because the compensation failed, so attempting the "mark as complete" request again might only lead to yet another error (such as *Ticket already marked as complete*). Talk about confusion on the part of the end user!"

"Yeah, I can imagine the developers coming to us to ask us how to resolve this issue," Addison said.

"Often developers are good checks on incomplete or confusing architecture solutions. If they are confused, there may be a good reason," said Logan. "OK, one more issue. Atomic

distributed transactions and corresponding compensating updates also impact responsiveness. If an error occurs, the end user must wait until all corrective action is taken (through compensating updates) before a response is sent telling the user about the error."

"Isn't that where the change to eventual consistency helps, for responsiveness?" asked Austen.

"Yes, while responsiveness can sometimes be resolved by asynchronously issuing compensating updates through eventual consistency (such as with the Parallel Saga and the Anthology Saga pattern), nevertheless most atomic distributed transactions have worse responsiveness when compensating updates are involved."

"OK, that makes sense—atomic coordination will always have overhead," Austen said.

"That's a lot of information. Let's build a table to summarize some of the trade-offs associated with atomic distributed transactions and compensating updates." (See Table 12-12.)

TRADE-OFFS

Table 12-12. Trade-offs associated with atomic distributed transactions and compensating updates

Advantages	Disadvantages
All data restored to prior state	No transaction isolation
Allows retries and restart	Side effects may occur on compensation
	Compensation may fail
	Poor responsiveness for the end user

Logan said, "While this compensating transaction pattern exists, it also offers a number of challenges. Who wants to name one?"

"I know: *a service cannot perform a rollback*," said Austen. "What if one of the services *cannot* successfully undo the previous operation? The orchestrator must have coordination code to indicate that the transaction wasn't successful."

"Right—what about another?"

"*To lock or not lock participating services?*" said Addison. "When the mediator places a call to a service and it updates a value, the mediator will make calls to subsequent services that are part of the workflow. However, what happens if another request appears for the first service contingent on the outcome of the first request's resolution, either from the same mediator or a different context? This distributed architecture problem becomes worse when the calls are asynchronous rather than synchronous (illustrated in "Phone Tag Saga(sac) Pattern"). Alternatively, the mediator could insist that other services don't accept calls during the course of a workflow, which guarantees a valid transaction but destroys performance and scalability."

Logan said, "Correct. Let's get philosophical for a moment. Conceptually, transactions force participants to stop their individual worlds and synchronize on a particular value. This is so easy to model with monolithic architectures and relational databases that architects overuse transactions in those systems. Much of the real world isn't transactional, as observed in the famous essay by Gregor Hohpe, "Starbucks Does Not Use Two-Phase Commit". Transactional coordination is one of the hardest parts of architecture, and the broader the scope, the worse it becomes."

"Is there an alternative to using an Epic Saga?" Addison asked.

"Yes!" Logan said. "A more realistic approach to the scenario described in Figure 12-24 might be to use either a Fairy Tale Saga or a Parallel Saga pattern. These sagas rely on asynchronous eventual consistency and state management rather than atomic distributed transactions with compensating updates when errors occur. With these types of sagas, the user is less impacted by errors that might occur within the distributed transaction, because the error is addressed behind the scenes, without end-user involvement. Responsiveness is also better with the state management and eventual consistency approach, because the user does not have to wait for corrective action to be taken within the distributed transaction. If we have issues with atomicity, we can investigate those patterns as alternatives."

"Thanks—that's a lot of material, but now I see why the architects made some of the decisions in the new architecture," Addison said.

Previous chapter

< 11. Managing Distributed Workflows

Next chapter

13. Contracts >